

Trabajo Fin de Título

Desarrollo de Aplicación Android para la comparación de precios en supermercados

Ciclo Formativo de Grado Superior en Desarrollo de
Aplicaciones Multiplataforma (DAM) 2º Curso

Autor: Savu, Raluca Alexandra
Tutor: Oliver Cano, Miguel

2º Curso DAM – Periodo Marzo/Junio 2025

Agradecimientos

Gracias a mi tutor de proyecto **Miguel Oliver Cano**,
por escuchar mis dudas a cualquier hora y ayudarme a mantener la motivación hasta el final. Ha sido un apoyo maravilloso.

Gracias a mi tutor **Alfredo Oltra Orenga**,
por apoyarme en todo, creer en mí hasta el final y no permitirme rendirme ni tirar la toalla.

Gracias a mi tutora de prácticas **Mara Vaño Alonso**,
por escucharme y aconsejarme a lo largo de este nuevo recorrido profesional.

Resumen

Este proyecto nace como respuesta a una necesidad que afecta cada vez a más personas: la dificultad de acceder a una alimentación asequible y de calidad en un contexto marcado por el encarecimiento de los productos básicos. Para hacer frente a esta situación, he desarrollado una aplicación móvil para dispositivos Android que permite comparar precios entre distintos supermercados, con el fin de ayudar a las familias a tomar decisiones informadas y ahorrar en sus compras cotidianas.

La aplicación está pensada para ser práctica y accesible. Permite escanear código de barras o buscar productos por nombre, comparar precios por unidad, crear listas de la compra, consultar el historial de precios y descubrir los supermercados más cercanos a nosotros.

El desarrollo se ha realizado con Android Studio en Java, y está concebido como un proyecto de código abierto. No busca el beneficio económico, sino generar un impacto social positivo: ofrecer una herramienta útil colaborativa para quienes más lo necesitan, fomentando la transparencia y el consumo responsable.

Abstract

This project was born in response to a growing need affecting more and more people: the difficulty of accessing affordable, high-quality food in a context marked by rising basic-goods prices. To address this situation, I have developed a mobile app for Android devices that allows users to compare prices across different supermarkets, helping families make informed decisions and save on their everyday shopping.

The app is designed to be practical and accessible. It lets you scan barcodes or search for products by name, compare unit prices, create shopping lists, view price history, and discover the nearest supermarkets.

The development was carried out in Android Studio using Java, and the project is open source. Its goal isn't economic profit, but rather to create a positive social impact: to offer a collaborative, useful tool for those who need it most, promoting transparency and responsible consumption.

Resum

Este projecte naix com a resposta a una necessitat que afecta cada vegada més persones: la dificultat d'accedir a una alimentació assequible i de qualitat en un context marcat per l'encariment dels productes bàsics. Per fer front a aquesta situació, he desenvolupat una aplicació mòbil per a dispositius Android que permet comparar preus entre diferents supermercats, amb la finalitat d'ajudar les famílies a prendre decisions informades i estalviar en les seues compres quotidianes.

L'aplicació està pensada per a ser pràctica i accessible. Permet escanejar codis de barres o cercar productes per nom, comparar preus per unitat, crear llistes de la compra, consultar l'historial de preus i descobrir els supermercats més propers.

El desenvolupament s'ha realitzat amb Android Studio en Java, i està concebut com un projecte de codi obert. No cerca el benefici econòmic, sinó generar un impacte social positiu: oferir una eina útil i col·laborativa per a qui més ho necessita, fomentant la transparència i el consum responsable.

Tabla de contenido

Agradecimientos	1
Resumen	2
Abstract	3
Resum	4
Tabla de contenido	5
Tabla de Ilustraciones	7
1. Introducción.....	9
1.1. Contexto y motivación del proyecto.....	9
1.2. Problema por resolver y necesidades detectadas	9
1.3. Objetivos generales y específicos	10
2. Estado del arte	11
2.1. Estudio del mercado	11
2.2. Aportación	13
3. Análisis de requisitos	14
3.1. Identificación de los usuarios finales.....	14
3.2. Requisitos funcionales y no funcionales	14
3.3. Restricciones y consideraciones técnicas	15
4. Tecnologías/ Herramientas utilizadas	16
5. Estudio de viabilidad.....	17
5.1. Análisis técnico, económico, operativo y legal	17
5.2. Análisis DAFO.....	18
5.3. Modelo de negocio	20
5.4. Supermercados elegidos	20
6. Planificación del proyecto	21
6.1. Diagrama de GANT inicial.....	21
7. Diseño	22
7.1. Arquitectura general de la aplicación.....	22
7.2. Diagramas de Casos de Uso.....	24
7.3. Diagrama entidad-relación.....	30
7.4. Diseño de interfaz de usuario (wireframes)	34
8. Desarrollo e Implementación	36
8.1. Activities y navegación entre ventanas.....	36
8.2. Firebase Authentication y perfil de usuario	39
8.3. Datos de usuario (favoritos e historial).....	42

8.4.	Carga de datos y adaptadores	44
8.5.	Lectura de datos: obtención de productos	46
8.6.	Escritura de datos.....	47
8.7.	Lógica de comparación de precios	48
8.8.	Funcionalidad del lector de código de barras	49
9.	Resultados finales	51
9.1.	Pantallas iniciales y de autenticación	51
9.2.	Página de inicio y perfil de usuario	51
9.3.	Pantalla de búsqueda y detalles de producto	52
9.4.	Pantallas de favoritos, actualización de precio.....	52
9.5.	Pantalla de añadir nuevo producto	53
		53
10.	Conclusiones y Trabajo Futuro	54
10.1.	Evaluación del grado de cumplimiento de los objetivos	54
10.2.	Análisis de la Planificación	54
10.3.	Lecciones aprendidas durante el desarrollo	54
10.4.	Posibles mejoras y líneas de trabajo futuras.	55
11.	Bibliografía y Referencias.....	56
12.	Anexo	57

Tabla de Ilustraciones

Ilustración 1 - Gráfica de ganancias supermercados [1]	9
Ilustración 2 - Título noticia sobre reduflación [2]	10
Ilustración 3 - Noticia sobre comparadores de precios [3]	10
Ilustración 4 - Ventana principal de Comparaloo	11
Ilustración 5 - Ventana de la aplicación SoySuper	11
Ilustración 6 - Capturas de pantalla de la aplicación Findit	12
Ilustración 7 - Logos de las herramientas utilizadas.....	16
Ilustración 8 - Ilustración del análisis DAFO	19
Ilustración 9 - Logos supermercados	20
Ilustración 10 - Diagrama GANTT inicial	21
Ilustración 11 - Diagrama de Casos de uso "Buscar Producto"	29
Ilustración 12 - Diagrama de Casos de uso "Actualizar Producto".....	29
Ilustración 13 - Estructura de la colección de productos de Firestore	30
Ilustración 14 - Estructura de la colección de usuarios en Firestore	31
Ilustración 15 - Estructura de las colecciones de supermercados, tipos y marcas	31
Ilustración 16 - Estructura de los productos - supermercados en Firestore	32
Ilustración 17 - Diagrama Entidad-Relación	32
Ilustración 18 - Vista de la consola de la colección de productos en Firestore	33
Ilustración 19 - Vista de la consola de Firestore de la colección de supermercados.....	33
Ilustración 20 - Wireframes del acceso a la app.....	34
Ilustración 21 - Wireframe de la búsqueda de un producto	34
Ilustración 22 - Wireframe de favoritos, actualizar precio y añadir producto nuevo.	35
Ilustración 23 - Vista de las carpetas de las Activities y los xmls correspondientes	36
Ilustración 24 - Screenshot del método onCreate().....	37
Ilustración 25 - Código que enseña el funcionamiento de un Intent.....	37
Ilustración 26 - Diagrama de flujo de navegación entre ventanas	38
Ilustración 27 - Muestra de código para el inicio de sesión.....	39
Ilustración 28 - Código del Registro del usuario en Auth.....	40
Ilustración 29 - Guardar info del usuario en Firestore.....	40
Ilustración 30 - Diagrama de flujo del registro.....	41
Ilustración 31 - Método de comprobar estado de favoritos.....	42
Ilustración 32 - Metodo de añadir a favoritos	42
Ilustración 33 - Método de quitar un producto de favoritos	43
Ilustración 34 - Diagrama de flujo de los favoritos.....	43
Ilustración 35 - Método de buscar producto	44
Ilustración 36 - Código del productAdapter	44
Ilustración 37 - Diagrama de flujo de la búsqueda de un producto	45
Ilustración 38 - Método de búsqueda de producto.....	46
Ilustración 39 - Método de guardar nuevo producto en la base de datos.....	47
Ilustración 40 - Método de calcular precio por supermercado	48
Ilustración 41 -XML de lector de código de barras.....	49
Ilustración 42 - Método que procesa el lector de código de barras	50
Ilustración 43 - Pantallas finales de autenticación.....	51

Ilustración 44 - Pantalla inicial y perfil de usuario	51
Ilustración 45 - Pantallas de búsqueda de producto	52
Ilustración 46 - Pantallas de favoritos y actualización de precio	52
Ilustración 47 - Pantallas de nuevo producto.....	53
Ilustración 48 - Diagrama de Gantt Final	57

1. Introducción

1.1. Contexto y motivación del proyecto

He vivido en primera persona las dificultades que muchas familias enfrentan para acceder a productos básicos debido al alto coste de la vida. La **inflación**, la **desigualdad** y la falta de transparencia en los precios han hecho que llenar la cesta de la compra sea cada vez más complicado, especialmente para quienes se encuentran en situaciones vulnerables. Mientras tanto, las grandes superficies siguen obteniendo **beneficios** desmesurados.

Esta situación me ha generado una profunda preocupación social y me ha motivado a desarrollar una herramienta que contribuya a cambiar esta realidad, aunque sea a pequeña escala. Por eso, decidí crear una aplicación que facilite a las personas comprar precios y tomar decisiones de compra más informadas, desde un enfoque ético, colaborativo y sin ánimo de lucro.



1.2. Problema por resolver y necesidades detectadas

- Falta de transparencia en los precios de productos alimenticios
- Dificultad para identificar el supermercado más barato para un producto determinado
- Ausencia de herramientas colaborativas y éticas que promuevan el consumo responsable
- Escasez de apps gratuitas y sin publicidad que cubran esta necesidad real.

La reduflación resiste sin ley: menos cantidad, más debate y cero transparencia

Las compañías han desarrollado su creatividad para camuflar lo esencial: que el consumidor paga lo mismo por menos

Ilustración 2 - Título noticia sobre reduflación [2]

1.3. Objetivos generales y específicos

Objetivos generales:

Desarrollar una aplicación móvil colaborativa que permite comparar precios de productos entre supermercados, brindando a los usuarios una herramienta ética y gratuita para tomar decisiones informadas de compra.

Objetivos específicos:

- Permitir la búsqueda de productos por nombre y por código de barras.
- Visualizar comparativas y evoluciones de precios.
- Implementar funcionalidades para registrar usuarios, favoritos e histórico de búsqueda.
- Fomentar un modelo colaborativo y open-source para el mantenimiento y evolución de la app.



Ilustración 3 - Noticia sobre comparadores de precios [3]

2. Estado del arte

2.1. Estudio del mercado

En el panorama actual existen varias aplicaciones que permiten comparar precios de productos entre supermercados, aunque muchas se centran en funciones básicas o están ligadas a intereses comerciales. A continuación, se presentan tres de las más representativas:

Comparaloo

Comparaloo [4] es un comparador web que agrupa los precios en tiempo real de supermercados españoles (Mercadona, Carrefour, Lidl, Dia...). Permite buscar productos, aplicar filtros por categoría, marca o precio y ordenar resultados. No requiere registro y es accesible en móvil y escritorio. Además, ofrece un blog con consejos y guías de ahorro.

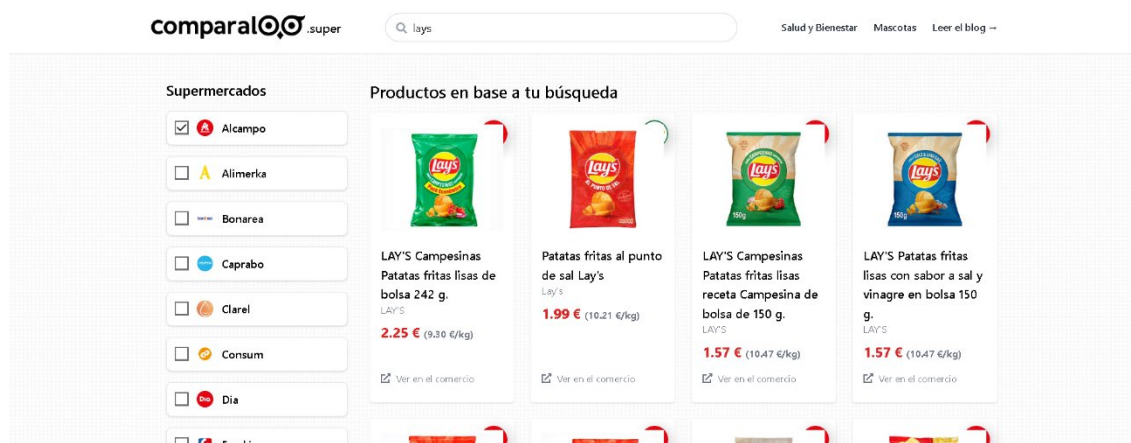


Ilustración 4 - Ventana principal de Comparaloo

SoySuper

Comparador de precios [5] que permite a los usuarios crear listas de compra en su supermercado habitual, incluyendo 9 supermercados de España. La aplicación cuenta con más de 130,000 productos con fotos, facilitando la planificación de compras.

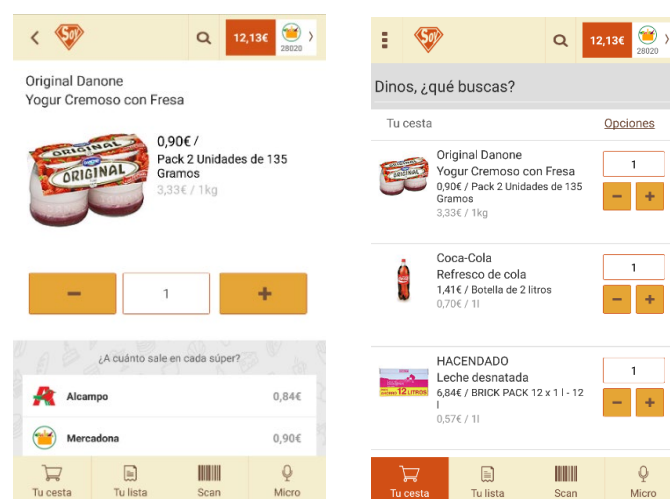


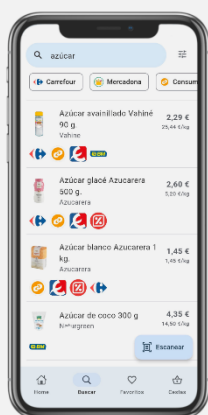
Ilustración 5 - Ventana de la aplicación SoySuper

Findit

Comparador de precios [6] multisectorial que actualmente permite comparar precios de productos en 8 supermercados y gasolineras en toda España. Los usuarios pueden buscar productos por nombre o escaneando su código de barras, y guardar productos en favoritos o en su cesta para compararlos fácilmente.

Encuentra lo que te interesa

Busca cualquier producto o escanea su código de barras y te decimos dónde puedes encontrarlo al **mejor precio**



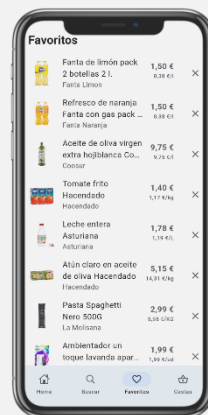
Explora productos y establecimientos

En **home** te mostramos los **productos y establecimientos** de mayor interés. También podrás ver las mejores **ofertas**



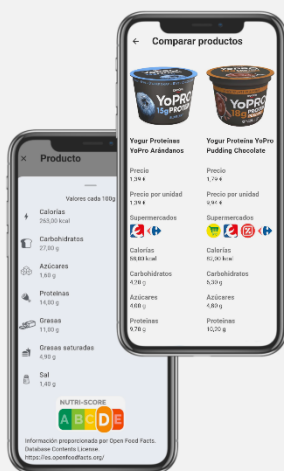
Guarda tus productos favoritos

Guarda en **favoritos** tus productos de **interés** y lleva un seguimiento de sus precios.



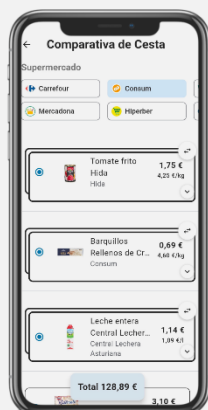
¿Valores nutricionales?

También los puedes ver con Findit. Descubre el **valor nutricional** y haz **comparativas** dos a dos de tus productos favoritos.



¡Compara cestas en distintos supermercados!

Nosotros te **compararemos** el precio de tu **cesta** en distintos supermercados y te diremos en qué súper está más **barata**



Utiliza el modo **Lista de la Compra** para poder ir tachando los productos que vayas comprando en tu supermercado. ¡Olvídate de los papelitos con letra que no se entiende y de notas cutres en tu móvil!

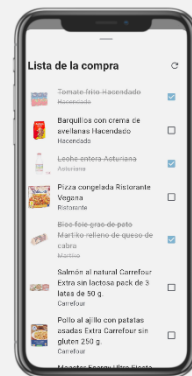


Ilustración 6 - Capturas de pantalla de la aplicación Findit

2.2. Aportación

Código abierto y sin ánimo de lucro

- A diferencia de soluciones como Comparaloo o SoySuper, cuyo desarrollo responde a intereses comerciales o modelos de negocio basados en comisiones y publicidad, nuestra herramienta se ofrece bajo una licencia open source.
- El repositorio estará disponible públicamente para que estudiantes y recién graduados —habituales usuarios de Java y Android Studio— puedan contribuir con nuevas funcionalidades, mejoras de interfaz o módulos de análisis de datos.
- Esta transparencia en el código respalda la confianza de la comunidad y facilita auditorías independientes sobre el correcto tratamiento de la información.

Sin publicidad ni comisiones ocultas

- Frente a apps como FindIt o SoySuper, que dependen de anuncios o acuerdos con supermercados para financiarse, nuestra plataforma garantiza un uso completamente gratuito y libre de cualquier banner publicitario.
- La financiación se basará exclusivamente en microdonaciones voluntarias y posibles subvenciones éticas de organizaciones sin ánimo de lucro, eliminando cualquier desviación en el orden de presentación de precios.
- Así, el usuario siempre verá los resultados más baratos primero, sin interferencias de patrocinadores ni promociones ocultas.

Datos colaborativos y verificados en tiempo real

- A diferencia de comparadores estáticos que actualizan precios una vez al día o con retrasos (por ejemplo, agrupadores webs como Comparaloo), nuestra app permitirá a los usuarios subir precios directamente desde la tienda mediante escaneo de código de barras o entrada manual.
- Esto no solo mejora la precisión de la información, sino que involucra a la comunidad en un proceso activo de construcción de valor colectivo.

3. Análisis de requisitos

3.1. Identificación de los usuarios finales

Los usuarios principales serán:

- Usuario registrado: puede consultar productos, añadirlos a favoritos, comparar precios, ver historial de búsqueda y consultar supermercados cercanos.
- Usuario moderador: puede hacer lo mismo que el usuario normal, además, puede actualizar los precios en los diferentes supermercados.
- Usuario administrador: puede hacer lo mismo que el moderador, además, puede añadir nuevos productos en la base de datos.
- Usuario sin registrarse: solo puede buscar productos por nombre o por código de barras.

3.2. Requisitos funcionales y no funcionales

Requisitos funcionales:

- **RF_01:** Registro de usuario.
- **RF_02:** Log in de usuario.
- **RF_03:** Búsqueda de producto por nombre.
- **RF_04:** Búsqueda por código de barras.
- **RF_05:** Comparador de precios.
- **RF_06:** Marcado del producto como favorito.
- **RF_07:** Mostrar evolución de precios en una gráfica.
- **RF_08:** Información de un producto (foto, nombre, marca, peso).
- **RF_09:** Los usuarios con rol de moderador pueden actualizar los precios de productos.
- **RF_10:** Solo los usuarios con rol de administrador pueden añadir nuevos productos a la base de datos.
- **RF_11:** El sistema debe permitir distinguir entre usuarios normales, administradores y moderadores mediante roles.
- **RF_12:** Solo los usuarios autenticados podrán actualizar sus datos o cerrar sesión.

Requisitos no funcionales:

- Interfaz intuitiva y accesible, adaptable a distintos tamaños de pantalla.
- Accesibilidad para personas con dificultades visuales (uso de etiquetas y contraste)
- Rendimiento fluido en dispositivos de gama media y baja
- Seguridad en la transmisión de datos y almacenamiento en la nube.
- Código abierto y disponible en un repositorio público (GitHub)
- Código modular y desacoplado, siguiendo principios SOLID.

3.3. Restricciones y consideraciones técnicas

Plataforma y versiones de Android:

- Versión mínima del SDK: API nivel 24 (Android 7.0)
- Versión objetivo del SDK: API nivel 35 (Android 13)

Tecnología y arquitectura

- Desarrollo en java 11 con Android Studio (2024.3), para aprovechar el soporte oficial y facilitar contribuciones
- Patrón arquitectónico MVVM para garantizar modularidad.
- Backend en Firebase (Authentication, Firestore), solución escalable y sin mantenimiento de servidor.

Gestión de dependencias y calidad de código

- Uso de Gradle con versiones fijas de librerías.

Licencia y contribución

- Licencia Open Source (MIT) que está declarada en el repositorio en un md.

UI/UX y guía de estilos

- Diseño siguiendo las 'Material Design Guidelines de Android' para garantizar coherencia y accesibilidad.

Coste y modelo de financiación

- App gratuita y abierta: equilibrio entre funcionalidad y consumo de recursos
- Soporte de donaciones voluntarias (como GitHub Sponsors, por ejemplo) sin integrar servicios de pago por defecto.

4. Tecnologías/ Herramientas utilizadas

El proyecto se apoya en un conjunto de herramientas y tecnologías seleccionadas por su robustez, comunidad activa y facilidad de integración en proyectos de código abierto:

- **Java:** [7] lenguaje de programación principal de la aplicación, elegido por su madurez, estabilidad y amplio ecosistema de bibliotecas. Su extenso soporte en Android Studio y la vasta documentación disponible facilitan el onboarding de nuevos contribuidores y garantizan compatibilidad a largo plazo.
- **Android Studio:** [8] entorno de desarrollo oficial para Android, que integra un editor visual de layouts, emuladores nativos, gestor de dependencias Gradle y herramientas de depuración. Ofrece plantillas y asistentes para acelerar la creación de componentes estándar, así como un sistema de análisis de rendimiento y linting que mejora la calidad del código.
- **XML:** [9] lenguaje de marcado utilizado para definir de manera declarativa las interfaces de usuario y gestionar recursos (layouts, estilos, temas y cadenas). Permite separar la capa de presentación de la lógica de negocio y centralizar la configuración visual, favoreciendo la mantenibilidad y la reutilización de componentes.
- **Firebase Authentication:** [10] gestión lista para usar de registro y login de usuarios (email/contraseña, Google, Facebook, anónimo) con verificación de correo, restablecimiento de contraseña y tokens seguros, todo integrado en el SDK de Android sin tener que montar nuestro propio sistema de credenciales.
- **Cloud Firestore:** [11] base de datos NoSQL en la nube con sincronización en tiempo real y cache offline, consultas flexibles (filtros, orden, paginación) y reglas de seguridad basadas en roles, que escala automáticamente y se factura según uso sin servidores propios.
- **ML Kit Barcode Scanner:** [12] API de escaneo de códigos de barras que funciona completamente en el dispositivo, sin requerir conexión a internet, lo que garantiza rapidez y privacidad en la lectura de códigos.
- **MPAndroidChart:** [13] Biblioteca para la creación de gráficos interactivos en Android, soportando diversos tipos de gráficos como líneas, barras y pasteles, con capacidad de escalado y animaciones.
- **GitHub:** [14] Plataforma de control de versiones y colaboración que permite la gestión del código fuente y la participación de la comunidad en el desarrollo del proyecto.
- **Glide:** [15] carga y cache de imágenes de forma eficiente, esencial para mostrar fotos de producto sin sobrecargar la memoria.
- **CameraX (AndroidX Camera):** [16] librería que unifica y simplifica la integración de la cámara en dispositivos Android de distintas versiones. Además, se integra de forma natural con ML Kit y con los componentes de Jetpack.

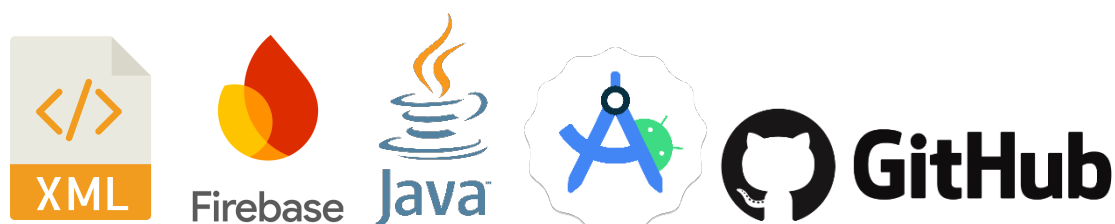


Ilustración 7 - Logos de las herramientas utilizadas

5. Estudio de viabilidad

5.1. Análisis técnico, económico, operativo y legal

Técnico:

El proyecto se basa en una arquitectura tecnológica accesible y ampliamente soportada:

- Plataforma principal: Android, debido a su gran cuota de mercado y compatibilidad con múltiples dispositivos.
- Backend y servicios en la nube: Firebase, que ofrece funcionalidades como base de datos en tiempo real, autenticación, hosting, almacenamiento y analítica sin necesidad de gestionar servidores propios.
- Lenguajes y tecnologías utilizadas: Java y XML para el desarrollo de la interfaz y lógica de la aplicación; GitHub como plataforma de control de versiones y colaboración.
- Complementos tecnológicos: integración futura con ML Kit para escaneo inteligente de etiquetas o código de barras, mejorando la usabilidad.

Económico:

El modelo económico se adapta a los principios del software libre y colaborativo:

- Durante las primeras fases del proyecto, se utilizarán herramientas gratuitas: Firebase, GitHub y otras utilidades de código abierto.
- En fases más avanzadas, donde haya mayor tráfico o necesidad de funciones premium de Firebase (como almacenamiento adicional en RealTime Database o consultas más intensivas), el coste estimado será de entre 5 y 10 euros mensuales.
- Se prevé un sistema de financiación ética y voluntaria, basado en donaciones a través de plataformas como GitHub Sponsors, Go FundMe, Kickstarter como campaña de crowdfunding inicial. Esto permite mantener el carácter gratuito y accesible de la aplicación.

Operativo:

El funcionamiento operativo estará centrado en una comunidad activa y comprometida:

- El proyecto será open-source, con código accesible públicamente para permitir la participación de desarrolladores de todo el mundo.
- Las actualizaciones de precios y productos se realizarán de forma colaborativa, permitiendo a los usuarios contribuir mediante un sistema moderado por voluntarios o usuarios verificados.
- Se establecerá una política de moderación transparente y colaborativa, similar a la de otros proyectos abiertos, para garantizar la veracidad de los datos y prevenir abusos.

Legal:

Desde el punto de vista legal, se ha evaluado cuidadosamente la viabilidad del proyecto:

- La aplicación no almacena datos personales sensibles más allá de la información mínima necesaria para el funcionamiento (como el correo electrónico si se usa autenticación, siempre bajo consentimiento del usuario).

- El sistema de recogida de precios se basa en la aportación de datos públicos o accesibles por los usuarios (como etiquetas físicas o páginas web de supermercados), lo que evita conflictos de propiedad intelectual o scrapping automático ilegal.
- Se adoptará una licencia de software libre, como la MIT [17], que permitirá el uso, modificación y redistribución del código, fomentando la transparencia y la mejora continua del proyecto.

5.2. Análisis DAFO

Fortalezas (internas):

- Propósito social claro: la app nace de una necesidad real, vivida por un gran número de personas, lo que refuerza la autenticidad y conexión con la sociedad.
- Open source y colaborativa: permite que cualquier desarrollador aporte funcionalidades sin ánimo de lucro.
- Gratuita y sin publicidad: accesible para todos, sin barreras económicas.
- Modelo ético y transparente; enfocado a luchar contra la avaricia de grandes superficies
- Flexibilidad en el desarrollo: al ser open source, puede evolucionar con el aporte de la comunidad.
- Sensibilidad social: promueve valores como la solidaridad, justicia y sostenibilidad.

Debilidades (internas):

- Falta de financiación directa: al no tener un modelo comercial, depende de donaciones o voluntariado.
- Dificultad en mantención de la infraestructura si no hay suficientes apoyos.
- Falta de recursos humanos constantes.
- Requiere mucho esfuerzo inicial para recopilar y mantener precios actualizados
- Menor visibilidad ante apps con inversión en marketing.

Oportunidades (externas)

- Creciente conciencia sobre consumo responsable y precios abusivos.
- Mayor sensibilidad social postpandemia y en épocas de crisis económica.
- Apertura a subvenciones, donaciones o plataformas como GitHub Sponsors.
- Espacio para integraciones con apps de recetas, nutrición o sostenibilidad.

Amenazas (externas)

- Competencia con apps similares con modelos comerciales fuertes (como Soysuper o Findit).
- Posible bloqueo de acceso a datos por parte de los supermercados.
- Desinterés general en colaborar si no se ve retorno económico.
- Dificultades legales o de privacidad si no se gestionan bien los datos.
- Cambios en normativas que impidan ciertas formas de recopilación de precios.



Ilustración 8 - Ilustración del análisis DAFO

5.3. Modelo de negocio

El modelo de negocio se basa en los principios del software libre y la colaboración comunitaria:

- **Open source:** El código fuente del proyecto estará disponible públicamente en GitHub, permitiendo la colaboración abierta y la transparencia en el desarrollo.
- **Modelo de Donaciones:** Se buscará financiación a través de donaciones voluntarias utilizando plataformas como GitHub Sponsors y campañas en sitios de crowdfunding como GoFundMe o Kickstarter.
- **Fondos iniciales:** Para cubrir los costes iniciales de infraestructura, se lanzarán campañas de crowdfunding, involucrando a la comunidad desde las primeras etapas del proyecto.
- **Creación de valor:** La aplicación proporcionará a los usuarios una herramienta gratuita y fácil de usar para comparar precios de productos, fomentando el ahorro y la transparencia en los precios.
- **Entrega de valor:** se ofrecerá una aplicación móvil gratuita, intuitiva y funcional, accesible para una amplia gama de usuarios.
- **Captura de valor:** la sostenibilidad del proyecto se basará en donaciones voluntarias de los usuarios y colaboradores que deseen apoyar el mantenimiento y desarrollo continuo de la aplicación.

5.4. Supermercados elegidos

He seleccionado Mercadona, Día y Alcampo para realizar la comparación de precios, cada uno por una razón diferente:

Mercadona:

- Lider de mercado: con la red de tiendas más amplia y establecida, es el referente al que acude la mayoría de consumidores.
- Marca propia: su apuesta por productos de marca blanca ofrece precios competitivos y uniformes, facilitando la comparación.

Día:

- Descuento y proximidad: tiendas de menor tamaño ubicadas en zonas urbanas, con rotación frecuente de ofertas y un enfoque claro en precios bajos.
- Accesibilidad: ideal para compras rápidas y para clientes que valoran la conveniencia sobre la variedad.

Alcampo:

- Hipermercado: combina alimentación con un extenso surtido de no alimentarios y compras a gran escala.
- Promociones por volumen: su modelo “compra al por mayor” y las campañas de acumulación de puntos ofrecen otra perspectiva sobre ahorro y coste unitario



Ilustración 9 - Logos supermercados

6. Planificación del proyecto

6.1. Diagrama de GANTT inicial

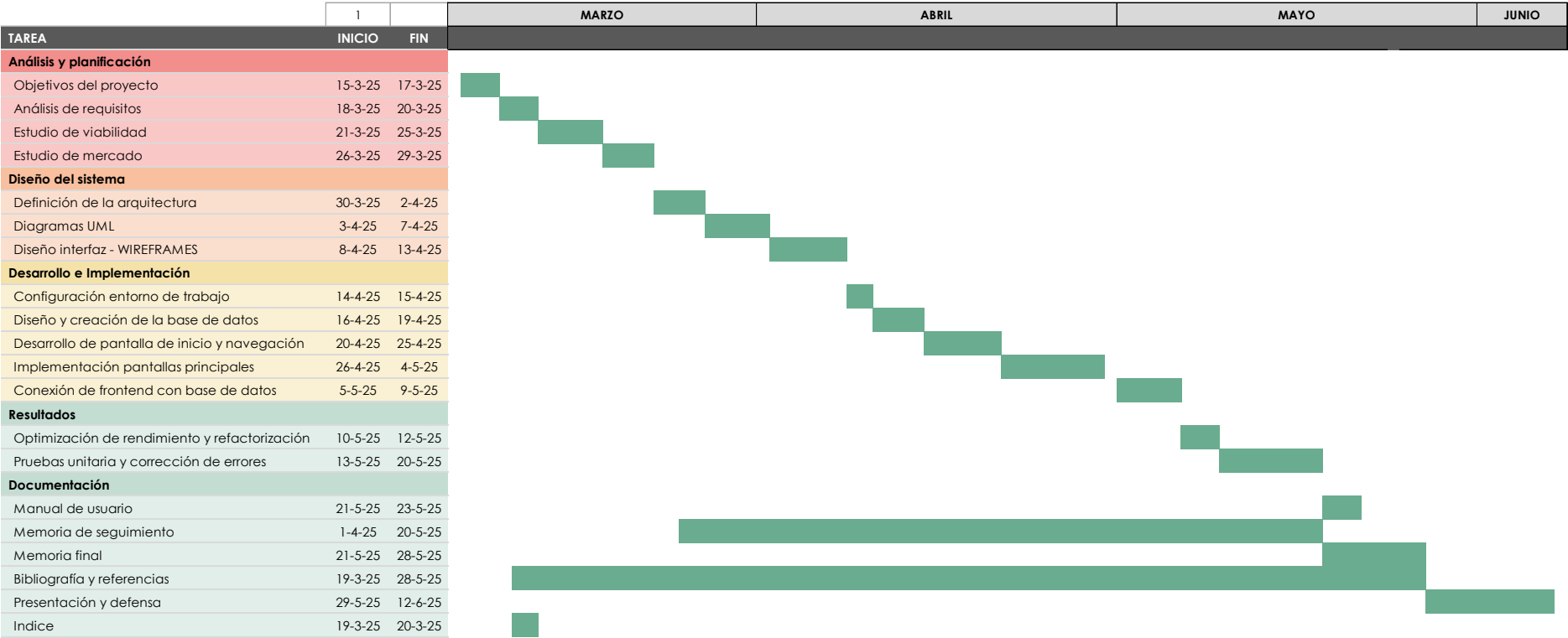


Ilustración 10 - Diagrama GANTT inicial

7. Diseño

7.1. Arquitectura general de la aplicación

Arquitectura utilizada

La aplicación sigue la arquitectura ‘clásica’ de Android basada en Activities como controladores de pantalla, complementada con los siguientes patrones y tecnologías:

- **UI Declarativa con XML + Activities:** cada pantalla implementa una Activity que enlaza un archivo de layout XML y gestiona la lógica mediante onCreate().
- **Navegación por Intents, Bottom Navigation y Top Bar:** se usa Intent para pasar entre Activities, un BottomNavigation para las secciones principales (Home, Escáner, Favoritos) y un TopBar para las secciones de ‘Añadir Producto’ y ‘Perfil de Usuario’.
- **Patrón Adaptador para Listas:** uso de RecyclerView junto con un Adapter y ViewHolder para presentar colecciones / vistas dinámicas de productos, favoritos o historial.
- **Acceso asíncrono a datos con callback:** todas las operaciones de lectura/escritura en Firebase Firestore y ML Kit son asíncronas, gestionadas con addOnSuccessListener/addOnFailureListener.
- **Backend en Firebase:**
 - Firebase Authentication para el registro, el login y gestión de sesiones.
 - Cloud Firestore para almacenar usuarios (con subcolecciones de favoritos e historial) productos, marcas, supermercados (con subcolecciones para las diferentes localizaciones) y precios por supermercado.
- **Bibliotecas auxiliares:**
 - Glide para cargar eficiente de imágenes desde URLs.
 - ML Kit + CameraX para escaneo de códigos de barras en tiempo real.
- **Gestión de permisos y servicios del sistema**
 - LocationManager para la obtención de la ubicación y cálculo de distancia a supermercados.
 - Manejo de permisos de cámaras y localización en tiempo de ejecución.

Componentes principales

- **Activities** (pantallas)
 - **MainActivity:** pantalla de bienvenida/ login –invitado
 - **LoginActivity / RegisterActivity:** Autenticación con Firebase Auth
 - **HomeActivity:** vista principal tras el login. Listado de historial, supermercados cercanos y búsqueda de productos.
 - **ProductDetailActivity/ ProductDetailFromBarcodeActivity:** detalle de productos, lógica de agregación/ actualización de precios y evolución histórica.
 - **FavouritesActivity:** listado de productos marcados como favoritos.
 - **AddProductActivity:** formulario para que los administradores creen nuevos productos.
 - **ProfileActivity:** visualización y edición de datos de usuario más el logout.
- **Layouts XML:**
 - Archivos de diseño por cada Activity (activity_*.xml).
 - Archivos de diseño para los items de listas como item_product.xml o item_nearby_supermarket.xml.

- Diseño para los menús `botom_navigation_menu.xml` y `top_app_bar_menu.xml`.
- **Navegación:**
 - Intents simples para pasar entre Activities.
 - `BottomNavigationView` integrado en la mayoría de los Activities para permitir el movimiento entre ventanas.
- **Modelos de datos**
 - Clase `Product` con anotaciones `Firestore` (`@DocumentId`, `@PropertyName`, `@Exclude`), campos básicos y calculados (precio mínimo, precio por unidad).
 - Clases `Brands`, `ProductTypes`, `Supermarkets` con campos básicos.
- **Adaptadores de RecyclerView**
 - `ProductAdapter` para enlazar `List<Product>` con vistas de listas reutilizables
 - `ViewHolder` inner class para optimizar bindings de vistas
- **Firestore:**
 - Colecciones: `users`, `products`, `brands`, `supermarkets`, `productSupermarket`, `productTypes`.
 - Sub-colecciones:
 - `Users`: `favouriteProducts` y `searchHistory`.
 - `Supermarkets`: `locations` con `GeoPoint`.
- **Firestore Authentication**
 - Manejo de cuentas con `FirebaseAuth.getInstance()`, métodos `signInWithEmailAndPassword`, `createUserWithEmailAndPassword`, `getCurrentUser()`, `signOut()`.
- **Escaneo de códigos de barras**
 - `CameraX PreviewView` + `ImageAnalysis` para capturar frames.
 - `ML Kit BarcodeScanner` configurado para formatos `EAN/UPC`, procesando `InputImage` y devolviendo el valor del código.
- **Lógica de agregación de precios**
 - Lectura de documentos en `productSupermarket` y por cada enlace, consulta a su subcolección `priceUpdate` ordenando por fecha para obtener el precio más reciente. Además, se calcula el precio mínimo y se recoge los logos de los supermercados.
- **Funcionalidad de ‘cerca de mí’**
 - `LocationManager` obtiene la última ubicación conocida.
 - Consulta a `supermarkets` y sus subcolecciones ‘locations’: calcula la distancia con `Location.distanceBetween()` seleccionando la sucursal más cercana por cadena y generando dinámicamente los items en el layout.

7.2. Diagramas de Casos de Uso

A continuación, detallaremos varios casos de uso de nuestra aplicación. Esta información se ha usado para crear los diagramas que veremos más adelante.

Caso_01: Buscar un producto por nombre

Actores principales: Usuario (registrado con rol usuario), Sistema

Requisitos: RF1, RF2, RF3, RF5, RF6, RF7, RF8

Descripción: Este caso de uso permite a un usuario identificado buscar un producto por nombre, consultar sus precios en distintos supermercados, visualizar detalles como el peso y la evolución del precio, añadirlo a favoritos y visualizar historial de búsqueda.

Precondiciones:

- El usuario debe estar autenticado
- La app debe tener acceso a Internet.
- Debe existir información del producto en Firebase

Flujo principal:

1. El usuario abre la app e inicia sesión
2. Visualiza la página principal: historial de búsqueda y supermercados cercanos
3. Accede a la barra de búsqueda y escribe el nombre del producto
4. El sistema sugiere coincidencias según el texto introducido
5. El usuario selecciona el producto deseado.
6. Se abre la pantalla de detalles del producto y el sistema le enseña:
 - a. Nombre completo
 - b. Peso del producto
 - c. Precio más barato (etiqueta 'Desde')
 - d. Lista de precios por supermercado
 - e. Precio por unidad de medida (€/kg, €/L, etc.)
 - f. Gráfica de evolución del precio
7. El usuario puede:
 - a. Ver los precios del producto en cada supermercado
 - b. Ver gráfico → media de los precios a lo largo del tiempo en todos los supermercados
 - c. Añadir el producto a favoritos

Flujos alternativos:

- Si no hay coincidencias, no se enseña nada.
- Si el producto es de marca blanca, se omite la comparación y solo se muestra un precio y el historial de precios.

Postcondiciones:

- El producto queda guardado como favoritos si el usuario lo selecciona

Componentes:

- Vista principal: barra de búsqueda y resultados

- Vista de detalle: pantalla con datos del producto
- ViewModel: coordina la búsqueda, obtención de datos y eventos de UI
- Repositorio: obtiene datos desde Firebase
- Firebase Firestore: proporciona información actualizada del producto, precios, historial y favoritos
- Lógica de negocio: calcula €/kg, ordena precios, gestiona datos para gráficas
- Componente gráfico: muestra la evolución del precio

Caso_02: Registro de un usuario

Actores principales: usuario (no registrado), sistema

Requisitos: RF_01

Descripción: Este caso de uso permite a una persona crear una cuenta de usuario en la aplicación. El registro puede hacerse mediante correo electrónico y contraseña.

Precondiciones:

- El usuario no debe tener una sesión iniciada.
- Debe estar habilitado el acceso a Firebase Authentication.
- El usuario debe tener acceso a internet haber dado permiso de recolección de la ubicación.

Flujo:

1. El usuario abre la app y pulsa sobre "Registrarse".
2. Elige el método de registro:
 - a. Correo electrónico y contraseña
3. Elige correo electrónico:
 - a. Introduce su email
 - b. Introduce una contraseña válida (ej. mínimo 6 caracteres)
 - c. Confirma la contraseña
4. El sistema valida los datos introducidos.
5. Si todo es correcto:
 - a. Crea el usuario en Firebase Authentication
 - b. Se guarda su perfil básico en la colección de usuarios de Firestore
6. Se inicia automáticamente la sesión y se redirige al usuario a la pantalla principal.

Flujos alternativos:

- Si el email ya está registrado, se muestra un mensaje de error.
- Si las contraseñas no coinciden, se solicita corregirlas.
- Si hay un error de conexión, se muestra un mensaje y se ofrece reintentar.

Postcondiciones:

- El usuario queda registrado y autenticado en la aplicación.
- Su perfil básico queda almacenado (correo, UID).

Componentes:

- UI de registro: permite introducir email y contraseña
- ViewModel: valida datos y comunica con el repositorio

- Firebase Auth: registra al usuario en la plataforma
- Firestore / SQLite: guarda información adicional del usuario
- UI principal: redirige tras el registro exitoso.

Caso_03: Buscar producto por código de barras

Actores principales: Usuario registrado (con rol de usuario normal), Sistema

Requisitos: RF_02, RF_04, RF_05, RF_06, RF_07, RF_08

Descripción: En este caso de uso, el usuario escanea el código de barras de un producto para buscar su información directamente y obtener los precios disponibles, características y evolución del precio.

Precondiciones:

- El usuario debe haber iniciado sesión.
- La app debe tener acceso a la cámara y a la ubicación.
- Debe tener acceso a Internet y acceso a Firebase.

Flujo:

- El usuario pulsa el botón de escanear en la vista principal.
- La app solicita permisos de cámara si no se han concedido.
- El usuario enfoca el código de barras.
- El sistema busca el producto en Firebase usando el código
- Si encuentra el producto:
 - a. Se abre la pantalla de detalles del producto (igual que el caso 01)
- El usuario puede:
 - a. Ver los precios por supermercado
 - b. Añadir a favoritos
 - c. Consultar evolución de precios en gráfica

Flujos alternativos:

- Si no se encuentra el código de barras, se muestra un mensaje de error diciendo que no se ha encontrado ese producto

Postcondiciones:

- El historial de visualizaciones se guarda en firestore.
- El producto puede ser añadir a favoritos.

Componentes:

- UI de pantalla de escaneo / introducción manual
- ViewModel: gestiona el escaneo e interacciona con Firebase.
- Firebase (Firestore): consulta el producto mediante el código
- Cámara (API Android): escaneo
- Lógica de negocio: calcula y ordena precios, prepara datos de visualización.

Caso_04: Actualizar el precio de un producto

Actores principales: Usuario registrado (con rol de 'mod' o 'admin'), Sistema

Requisitos: RF_09, RF_11

Descripción: este caso permite a usuarios registrados con permisos verificados (moderador o administrador) actualizar el precio de un producto visto en un supermercado en concreto.

Precondiciones:

- El usuario debe haber iniciado sesión y estar verificado.
- El producto debe existir previamente en la base de datos.
- La app debe tener acceso a Firebase.

Flujo:

1. El usuario accede a los detalles de un producto
2. Pulsa el icono de actualizar ubicado al lado del precio del supermercado.
3. Se abre un formulario donde introduce:
 - a. Precio actualizado
 - b. Fecha (se guarda automáticamente con el precio)
4. El sistema valida que el precio sea coherente (ej. No negativo, no muy desviado)
5. Se guarda el nuevo dato en el histórico del producto (en Firestore).

Flujos alternativos:

- Si el usuario no está verificado, se muestra un mensaje indicando que debe solicitar acceso para colaborar (para ser moderador).
- Si el precio es incoherente, salta un error y se le informa al usuario que hay una diferencia de más de 30% en el precio inicial.

Postcondiciones:

- El nuevo precio se añade al histórico del producto y puede usarse en las gráficas.

Componentes:

- UI de formulario para introducir precio.
- ViewModel: valida el dato y comunica con repositorio
- Firebase: almacena nuevo precio (colección de precios)
- Lógica de negocio: revisa coherencia, actualiza medias y gráficas.

Caso_05: Añadir un nuevo producto al sistema

Actores principales: usuario con rol de administrador, Sistema

Requisitos: RF_10, RF_11

Descripción: permite a un administrador agregar un nuevo producto a la base de datos, incluyendo su nombre, código de barras, categoría, peso, imagen etc.

Precondiciones:

- El actor debe haber iniciado sesión con permisos elevados
- Debe estar disponible la conexión con Firebase.

Flujo:

1. El administrador accede al menú de 'Nuevo Producto.
2. Pulsa 'añadir nuevo producto'.
3. Introduce los datos requeridos:
 - a. Nombre completo
 - b. Código de barras(único)
 - c. categoría (desplegable)
 - d. Marca (desplegable)
 - e. Peso
 - f. Imagen del producto (link a la imagen)
 - g. Precio
 - h. Supermercados que los lleven
4. El sistema valida que el código no esté duplicado
5. El producto se guarda en la base de datos

Flujos alternativos:

- Si el código ya existe, se muestra un mensaje y no se permite continuar.
- Si falta información obligatoria, se avisa y se evita el guardado.

Postcondiciones:

- El producto queda almacenado y disponible para los usuarios.
- El sistema lo incluye en futuras búsquedas y comparaciones.

Componentes:

- UI de formulario: pantalla de administrador.
- ViewModel: recoge los datos, valida y comunica con el repositorio
- Firebase Firestore: almacena datos del producto.

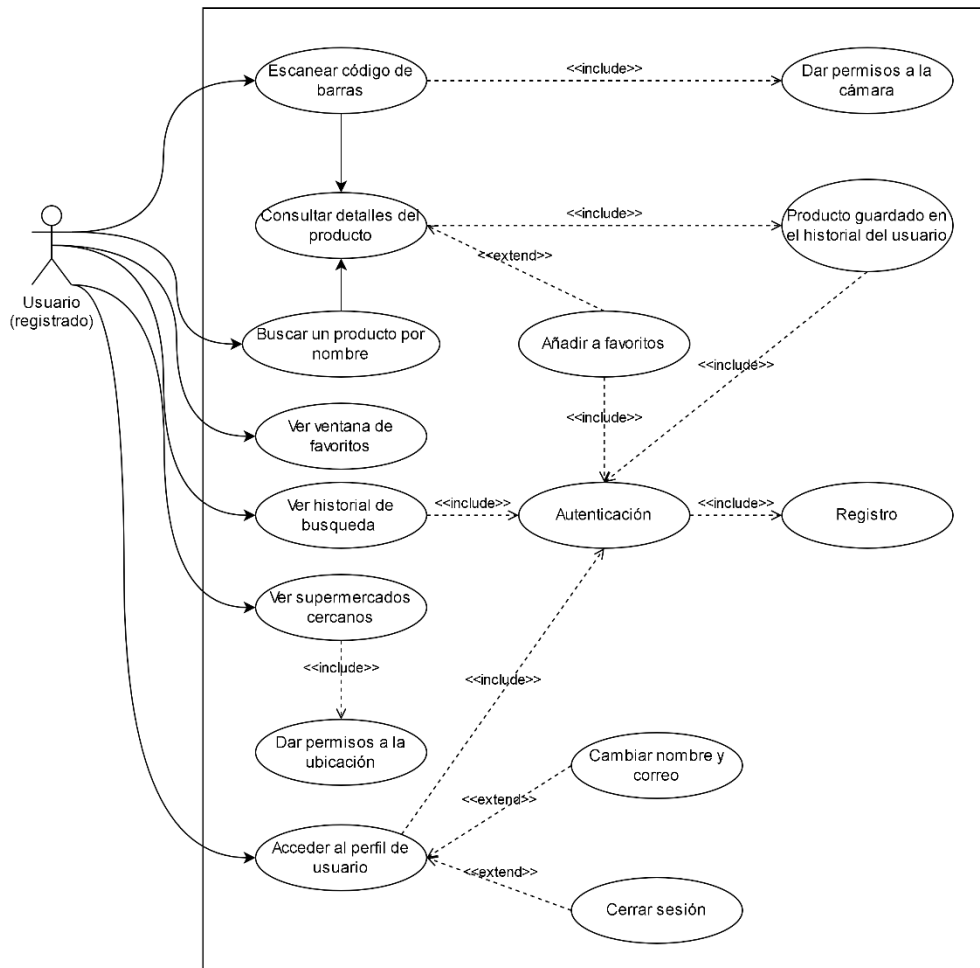


Ilustración 11 - Diagrama de Casos de uso "Buscar Producto"

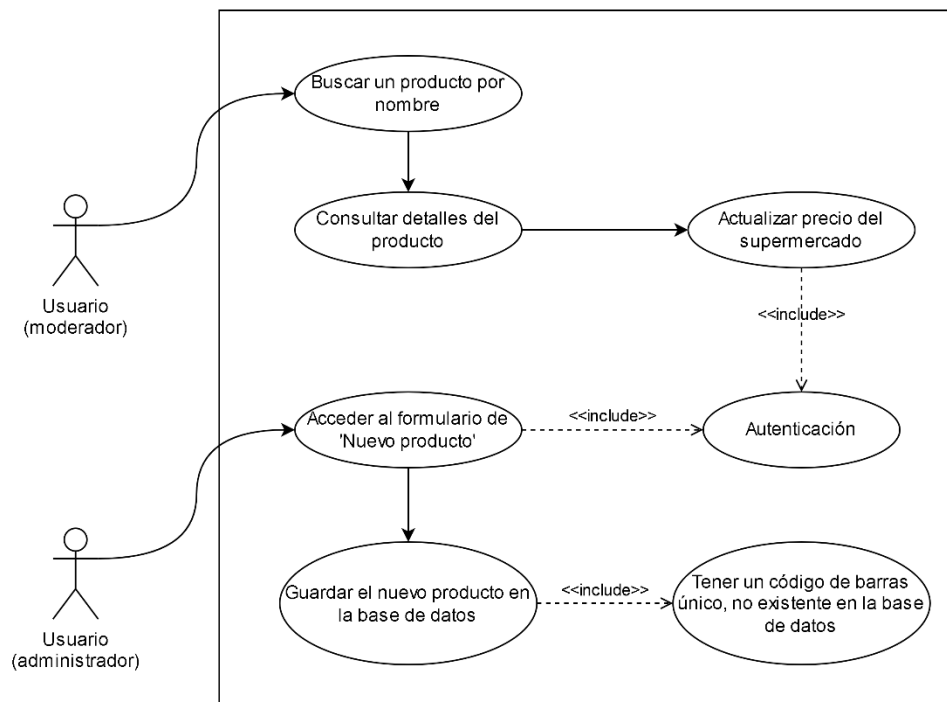


Ilustración 12 - Diagrama de Casos de uso "Actualizar Producto"

7.3. Diagrama entidad-relación

Para la base de datos utilizamos **Firestore**, una base de datos **NoSql** en la nube, proporcionada por Google. Firestore organiza los datos de forma jerárquica en colecciones, documentos y sub-colecciones

Colecciones: son como carpetas en un archivo. Cada colección agrupa elementos relacionados.

- **Ejemplo:** en nuestra app tenemos colecciones como products, users, brands, supermarkets, productTypes y productSupermarkets

Documentos: son los ‘archivos’ individuales dentro de una colección. Cada documento guarda los datos de un único ítem y tiene un ID único dentro de su colección.

- Ejemplo: en la colección ‘products’ hay un documento para ‘Leche Desnatada’. Cada documento tiene un ID único autogenerado por Firestore.

Campos: son los datos dentro de un documento, equivalente a los atributos del modelo de datos de ‘product’.

- Ejemplo: cada campo tiene un nombre (name, barCode, brand...) y un valor.

Subcolecciones: las subcolecciones son como colecciones con sus respectivos documentos pero dentro de un documento en concreto.

```
{
  "products": {
    "product_id_123": {
      "name": "Leche",
      "barCode": "1234567890123",
      "brandId": "brand_xyz",
      "photoUrl": "url.com",
      "productType": "productType_Id",
      "quantityPack": 6,
      "quantityUnity": 1.5,
      "unit": "kg"
    }
  },
}
```

Ilustración 13 - Estructura de la colección de productos de Firestore

```

"users": {
  "user_id_123":{
    "email": "email@gmail.com",
    "location": "geoPoint",
    "name": "Alexandra",
    "role": "admin",
    "favouriteProducts":{
      "favourite_Id_12":{
        "productId": "product_id_123"
      }
    },
    "searchHistory":{
      "history_Id_1223":{
        "productId": "product_id_123",
        "timeStamp": "timeStamp"
      }
    }
  }
},

```

Ilustración 14 - Estructura de la colección de usuarios en Firestore

```

},
"supermarkets":{
  "supermarket_id_12":{
    "name": "Día",
    "logoUrl": "url.com",
    "locations":{
      "location_id_12":{
        "address": "Calle Navarez 45",
        "location": "geoPoint",
        "name": "Día-Cale Navarez",
        "postalCode": 123456
      }
    }
  }
},
"productTypes":{
  "productType_id_123":{
    "name": "Conservas"
  }
},
"brands":{
  "brand_id_123":{
    "name": "Nestle"
  }
},

```

Ilustración 15 - Estructura de las colecciones de supermercados, tipos y marcas

```

},
"productSupermarket":{
  "prodSuper_id_123":{
    "productId": "product_id_123",
    "supermarketId": "supermarket_id_12",
    "priceUpdate":{
      "priceUpdate_Id_123":{
        "lastPriceUpdate": "timeStamp",
        "price": 12.54
      }
    }
  }
}
}
}
}
}

```

Ilustración 16 - Estructura de los productos - supermercados en Firestore

Entidad-Relación:

A continuación, se enseña un diagrama entidad relación para poder entender las relaciones entre colecciones y subcolecciones.

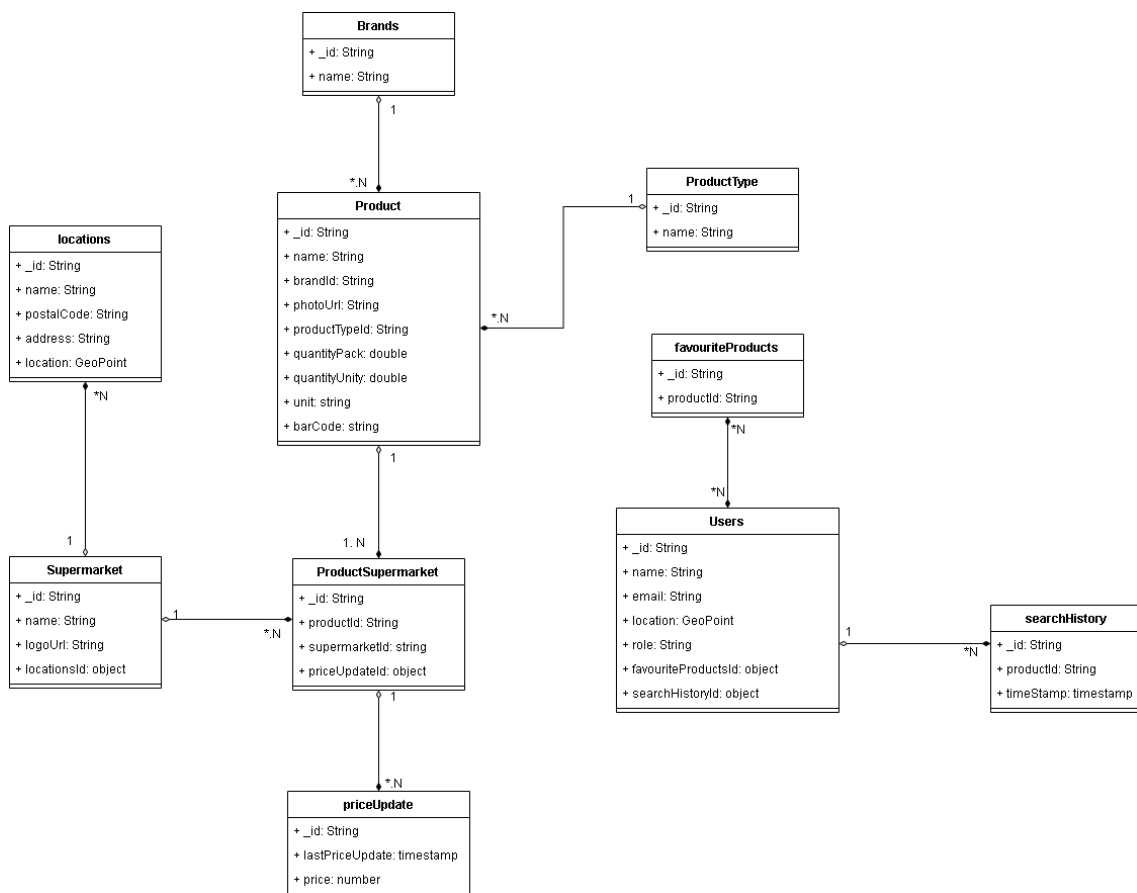


Ilustración 17 - Diagrama Entidad-Relación

Firestore:

También podemos ver como se vería la consola de Firestore desde donde se introducen los datos en la base de datos.

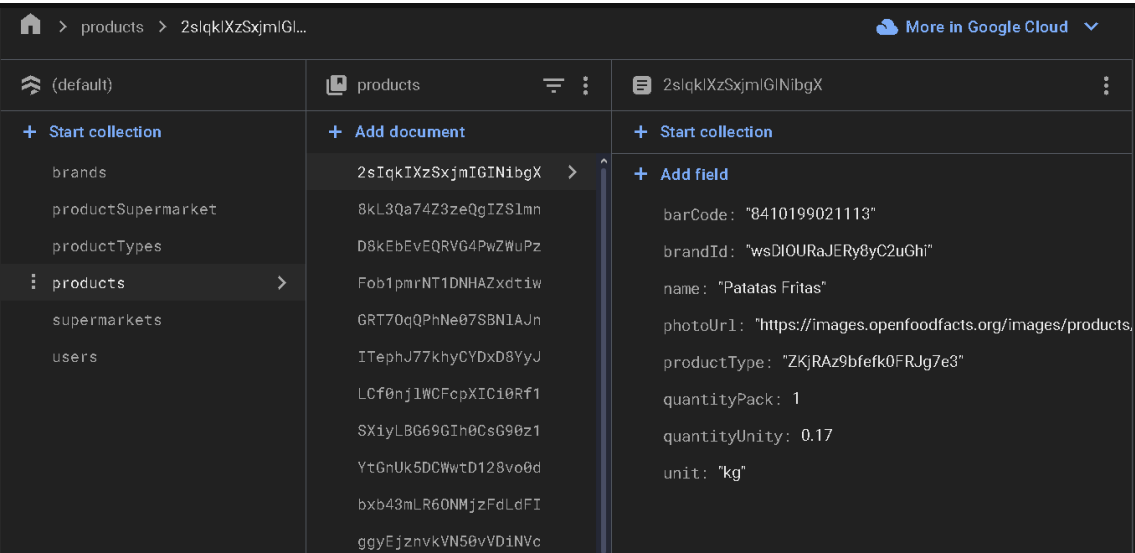


Ilustración 18 - Vista de la consola de la colección de productos en Firestore

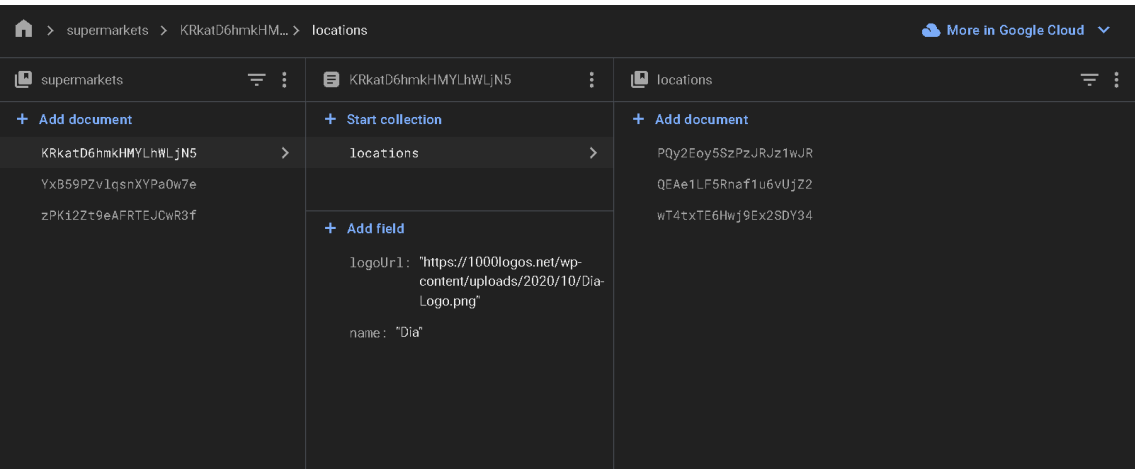


Ilustración 19 - Vista de la consola de Firestore de la colección de supermercados

7.4. Diseño de interfaz de usuario (wireframes)

En este apartado, veremos representado a modo de wireframes, ilustraciones del diseño propuesto para nuestra aplicación. Estas ilustraciones nos muestran una versión simplificada del flujo de trabajo de nuestra app.

Las primeras ilustraciones muestran las pantallas de acceso al sistema: el login y el registro de un usuario. En la primera pantalla el usuario puede escoger si iniciar sesión, registrarse o simplemente entrar como invitado. También veremos la pantalla donde el usuario puede modificar su información (menos el rol) o cerrar sesión.



Ilustración 20 - Wireframes del acceso a la app

Las siguientes ventanas nos muestra como se vería la página principal que se muestra nada más autenticarse. La página de inicio nos muestra un **historial de búsqueda** y los **supermercados** más cercanos. Todo esto si nos hemos autenticado y hemos compartido nuestra ubicación con la app. A continuación, vemos como sería si un usuario busca un producto. Le aparecerá una lista con todas las coincidencias y despues podrá escoger un producto, y ver sus detalles.

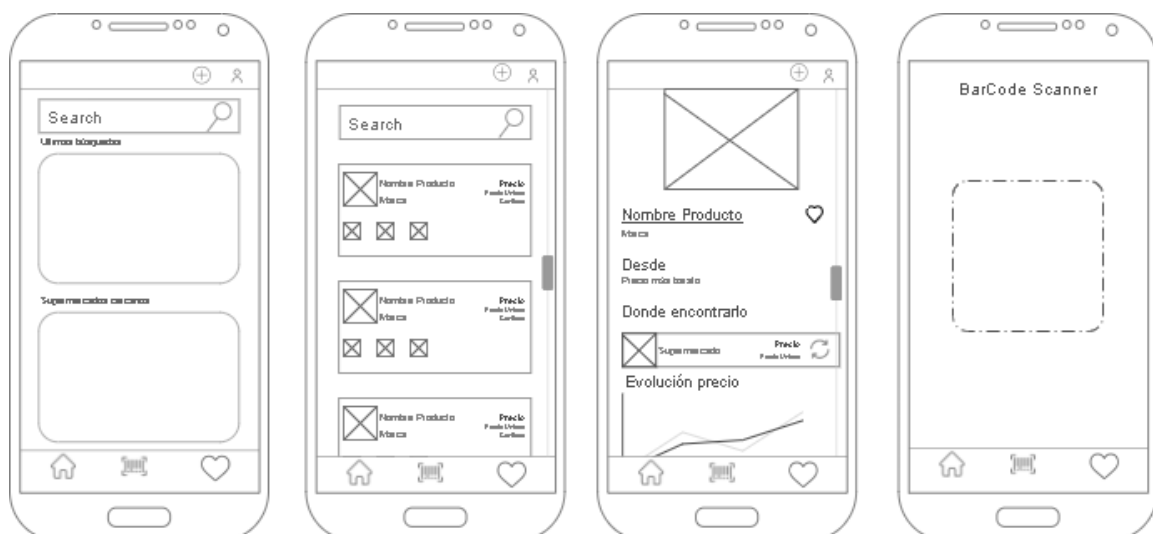


Ilustración 21 - Wireframe de la búsqueda de un producto

Las últimas ilustraciones muestran como se vería la página de favoritos de un usuario, como puede un moderador actualizar el precio de un productos y como puede un administrador añadir nuevos productos a la base de datos.



Ilustración 22 - Wireframe de favoritos, actualizar precio y añadir producto nuevo.

8. Desarrollo e Implementación

8.1. Activities y navegación entre ventanas

En nuestra aplicación se utiliza un modelo de Actividades de Android para cada pantalla, de modo que cada parte de la interfaz y su lógica quedan separadas:

- **Activities (Activity):** cada pantalla está definida por una clase Java donde implementamos toda la lógica de la interfaz: que ocurre al pulsar un botón, cómo se gestiona las peticiones a la base de datos, etc.
- **Diseño (Layout):** junto a cada Activity creamos un archivo XML de diseño. Ahí se decide la posición de botones, textos, imágenes y su aspecto.

Todo Activity tiene un archivo xml correspondiente, pero no todo .xml tiene un Activity equivalente. La razón la veremos más adelante cuando hablemos de RecyclerViews.

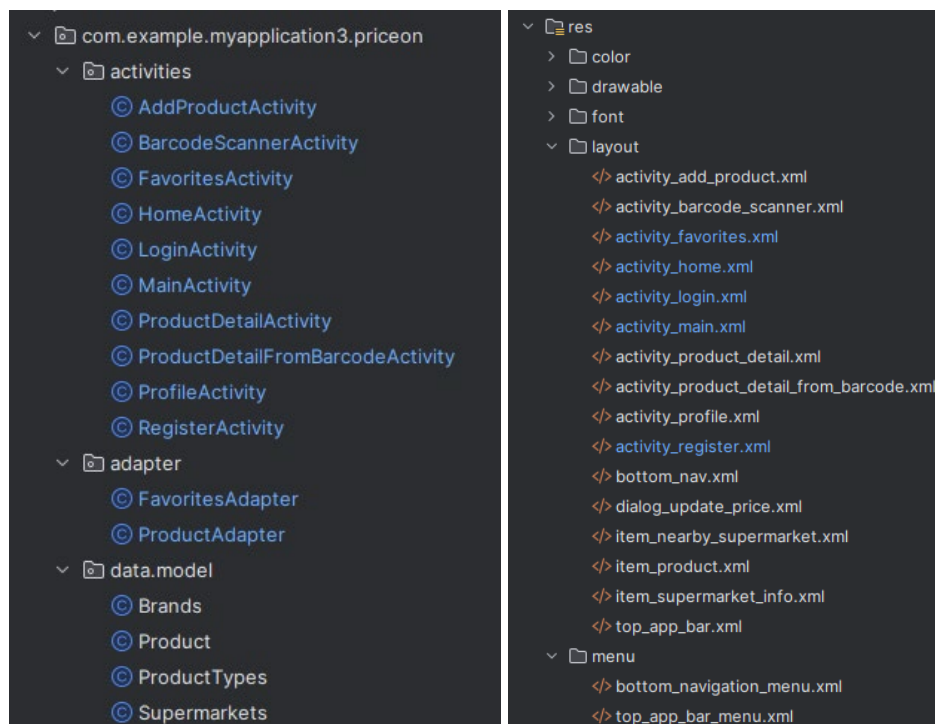


Ilustración 23 - Vista de las carpetas de las Activities y los xmls correspondientes

*Veremos a lo largo del código, de las nomenclaturas y de los nombres de los archivos, que hemos escogido nombrar todo en **inglés**. Eso se debe a varias razones:

- **Lengua oficial de la industria:** la mayoría de la documentación comparten este idioma, lo que facilita el buscar soluciones y compartir conocimientos.
- **Comunidad y colaboración:** esto facilita la colaboración con desarrolladores de todo el mundo, lo que hará que nuestra app pueda crecer continuamente.
- **Sigue uno de los principios del cleanCode.**

Método onCreate()

Dentro de todos los Activities, encontraremos este método, que es fundamental para su preparación / puesta en escena. Cuando Android muestra la pantalla, primero lo ejecuta y es ahí cuando se llama a **setContentView(R.layout.activity_main)** para cargar el diseño XML. Luego se usan **findViewById()** para enlazar los elementos de la interfaz con el código java.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    loginButton = findViewById(R.id.loginButton);
    registerButton = findViewById(R.id.registerButton);
    guestLoginTextView = findViewById(R.id.guestLoginTextView);
}
```

Ilustración 24 - Screenshot del método onCreate()

Movernos entre pantallas usando 'Intents'

Para poder movernos, por ejemplo, de la pantalla Main a la del Login cuando se pulsa el botón de 'Iniciar Sesión', utilizamos un **Intent**.

```
loginButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        startActivity(new Intent( packageContext: MainActivity.this, LoginActivity.class));
    }
});

registerButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        startActivity(new Intent( packageContext: MainActivity.this, RegisterActivity.class));
    }
});

guestLoginTextView.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        startActivity(new Intent( packageContext: MainActivity.this, HomeActivity.class));
    }
});
```

Ilustración 25 - Código que enseña el funcionamiento de un Intent

Desglosemos este fragmento de código por partes:

- **loginButton.setOnClickListener(...)**: Define lo que ocurre al pulsar el botón de “Login”.
- **Intent intent = new Intent(MainActivity.this, LoginActivity.class)**: Crea un Intent indicando desde dónde (MainActivity) y hacia dónde (LoginActivity) queremos ir.
- **startActivity(intent)**: Inicia la Activity indicada por el Intent.

Como funciona el flow de la Navegación

En el flujo que veremos a continuación muestra como el usuario interactúa con la pantalla actual (MainActivity), cuyo código crea un Intent que indica la siguiente pantalla (LoginActivity) y lo envía al sistema Android con **startActivity()**. El sistema recibe el Intent, determina la clase a arrancar, instancia LoginActivity y llama a su método **onCreate()**, donde **setContentView()** carga su diseño (**activity_login.xml**) y muestra la nueva pantalla.

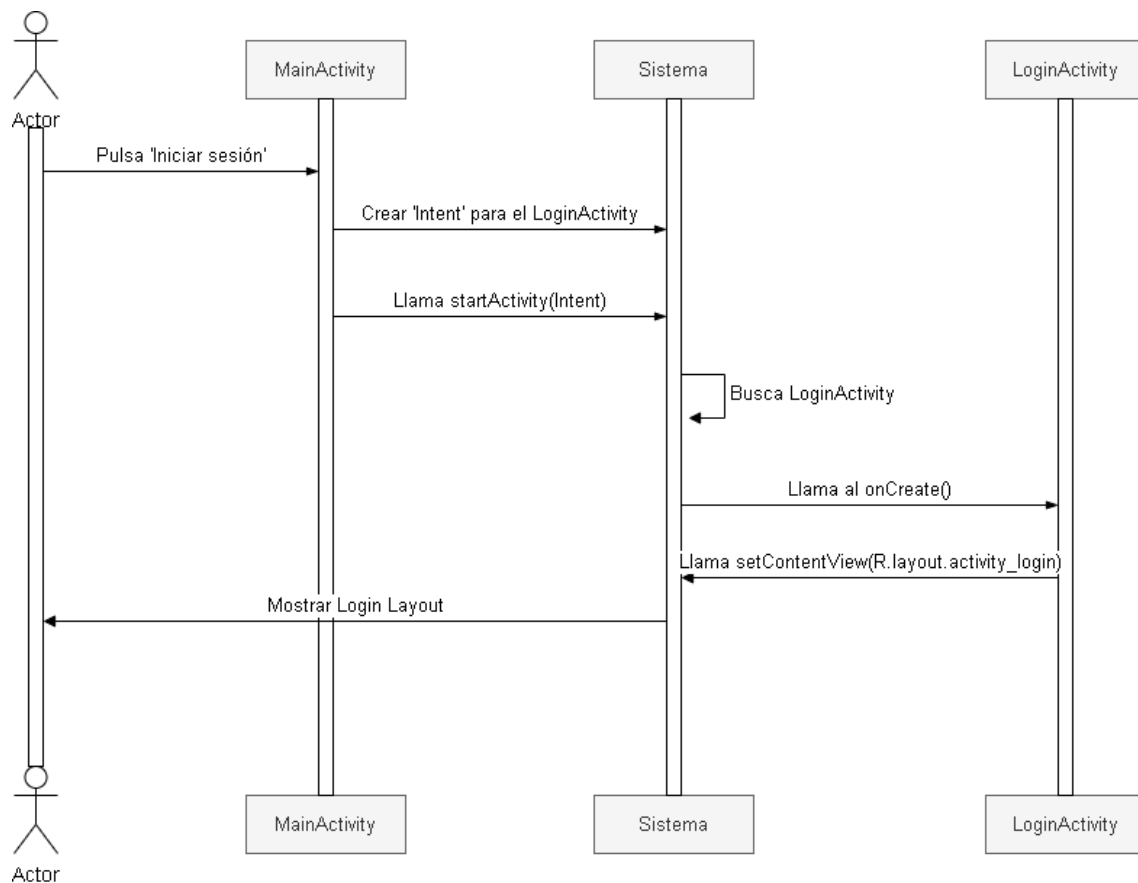


Ilustración 26 - Diagrama de flujo de navegación entre ventanas

8.2. Firebase Authentication y perfil de usuario

En nuestra app se utiliza Firebase Authentication para gestionar el inicio de sesión y el registro de usuarios, y Firebase Firestore para almacenar detalles adicionales de cada usuario, como su nombre, rol, su ubicación o su historial de búsquedas.

¿Porque Firebase Authentication?

Firebase Authentication ofrece un sistema seguro y listo para usar que se ocupa de toda la parte complicada del backend, permitiéndonos centrarnos en desarrollar las funcionalidades de la app.

Aunque soporta múltiples métodos de autenticación, nosotros nos centramos en el método más común de correo electrónico y contraseña.

Inicio de sesión

Cuando un usuario pulsa el “Log In” en LoginActivity sucede lo siguiente:

1. Se obtiene la instancia de `FirebaseAuth`.
2. Se leen el email y la contraseña de los campos de texto.
3. Se llama a `firebaseAuth.signInWithEmailAndPassword(email, password)`.
4. Con `addOnCompleteListener` se comprueba el resultado: si es exitoso, se navega a `HomeActivity`; si falla, se muestra un error al usuario.

```
1 usage
2 private void loginUser() {
3     String email = emailEditText.getText().toString().trim();
4     String password = passwordEditText.getText().toString().trim();
5
6     if (TextUtils.isEmpty(email) || TextUtils.isEmpty(password)) {
7         Toast.makeText(context, this, text: "Introduce email y contraseña", Toast.LENGTH_SHORT).show();
8         return; // Salimos de la función si no hay datos
9     }
10
11     //Iniciamos sesión con el método más común y simple de Firebase
12     firebaseAuth.signInWithEmailAndPassword(email, password)
13         .addOnCompleteListener( Task<AuthResult> task -> {
14             if (task.isSuccessful()) {
15                 Toast.makeText(context, this, text: "Inicio de sesión correcto", Toast.LENGTH_SHORT).show();
16                 startActivity(new Intent( packageContext LoginActivity.this, HomeActivity.class));
17                 finish();
18             } else {
19                 Toast.makeText(context, this, text: "Error al iniciar sesión: " + Objects.requireNonNull(task.getException()).getMessage(), Toast.LENGTH_LONG).show();
20             }
21         });
22 }
```

Ilustración 27 - Muestra de código para el inicio de sesión

Registro

El registro de usuario es un poco más complicado ya que implica el guardado de los datos en el firebase Auth pero también en la colección de usuarios del Firestore:

- Se obtienen instancias de `FirebaseAuth` y `FirebaseFirestore`.
- Llamamos a `mAuth.createUserWithEmailAndPassword(email, password)` para crear la cuenta en Firebase Auth.
- Si `task.isSuccessful()`, obtenemos al usuario con `mAuth.getCurrentUser()` y su UID.
- Preparamos un Map con datos como nombre, email, rol (“user”) y, opcionalmente, ubicación.
- Guardamos ese mapa en Firestore

A continuación, podemos ver el trozo de código del registro del usuario seguido del guardado de su información en Firestore, para posterior utilización.

```
public class RegisterActivity extends AppCompatActivity {
    private void registerUser() {
        String name = nameEditText.getText().toString().trim();
        String email = emailEditText.getText().toString().trim();
        String password = passwordEditText.getText().toString().trim();
        String confirmPassword = confirmPasswordEditText.getText().toString().trim();

        if (name.isEmpty() || email.isEmpty() || password.isEmpty() || confirmPassword.isEmpty()) {
            Toast.makeText(context, this, text: "Completa todos los campos", Toast.LENGTH_SHORT).show();
            return;
        }

        if (!password.equals(confirmPassword)) {
            Toast.makeText(context, this, text: "Las contraseñas no coinciden", Toast.LENGTH_SHORT).show();
            return;
        }

        if (password.length() < 6) {
            Toast.makeText(context, this, text: "La contraseña debe tener al menos 6 caracteres", Toast.LENGTH_SHORT).show();
            return;
        }

        //Creamos usuario en Firebase Auth
        firebaseAuth.createUserWithEmailAndPassword(email, password)
            .addOnCompleteListener(new Task<AuthResult> task -> {
                if (task.isSuccessful()) {
                    String uid = Objects.requireNonNull(firebaseAuth.getCurrentUser()).getUid();
                    // Guardar temporalmente para usar luego si se aprueba el permiso
                    pendingUid = uid;
                    pendingName = name;
                    pendingEmail = email;

                    if (ContextCompat.checkSelfPermission(context, Manifest.permission.ACCESS_FINE_LOCATION)
                        != PackageManager.PERMISSION_GRANTED) {
                        showLocationPermissionDialog();
                    } else {
                        saveUserToFirestore(uid, name, email);
                    }
                } else {
                    Toast.makeText(context, this, text: "Error: " + Objects.requireNonNull(task.getException()).getMessage(), Toast.LENGTH_LONG).show();
                }
            });
    }
}
```

Ilustración 28 - Código del Registro del usuario en Auth

```
4 usages
private void saveUserToFirestore(String uid, String name, String email) {
    LocationManager locationManager = (LocationManager) getSystemService(LOCATION_SERVICE);
    Location location = null;

    if (ActivityCompat.checkSelfPermission(context, Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED) {
        location = locationManager.getLastKnownLocation(LocationManager.GPS_PROVIDER);
    }

    GeoPoint geoPoint = location != null ? new GeoPoint(location.getLatitude(), location.getLongitude()) : null;

    Map<String, Object> userMap = new HashMap<>();
    userMap.put(k: "name", name);
    userMap.put(k: "email", email);
    userMap.put(k: "role", v: "user");
    userMap.put(k: "location", geoPoint);

    firestore.collection(collectionPath: "users").document(uid).set(userMap) Task<Void>
        .addOnSuccessListener(new Void aVoid -> {
            Toast.makeText(context, RegisterActivity.this, text: "Registro exitoso", Toast.LENGTH_SHORT).show();
            startActivity(new Intent(context, RegisterActivity.this, HomeActivity.class));
            finish();
        })
        .addOnFailureListener(new Exception e -> {
            Toast.makeText(context, RegisterActivity.this, text: "Error al guardar datos: " + e.getMessage(), Toast.LENGTH_LONG).show();
        });
}
```

Ilustración 29 - Guardar info del usuario en Firestore

A continuación, para entender un poco mejor el flujo del registro veremos un diagrama que plasma todo el proceso.

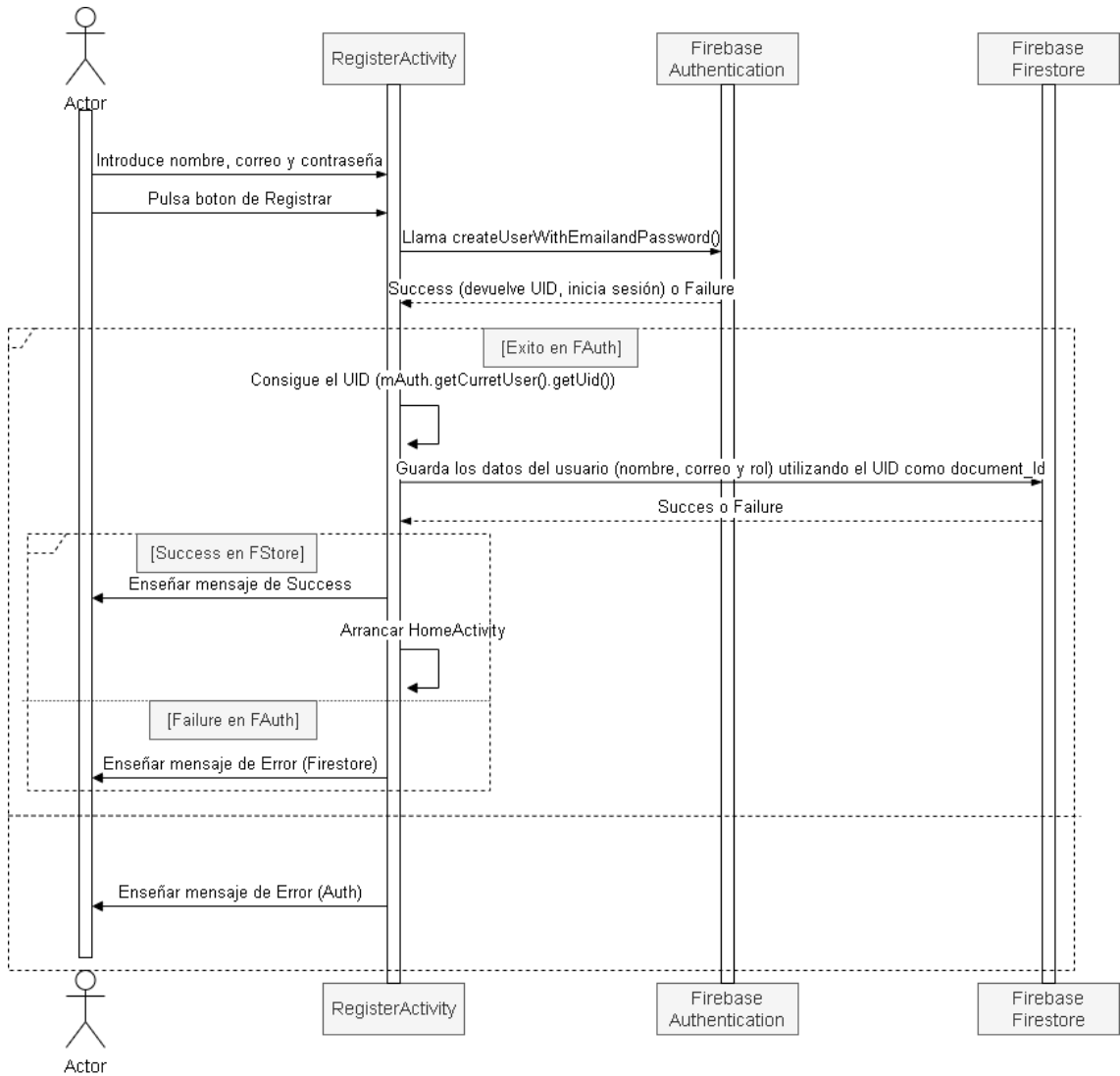


Ilustración 30 - Diagrama de flujo del registro

8.3. Datos de usuario (favoritos e historial)

Añadir / Quitar productos de favoritos

En el ProductDetailActivity, al pulsar el icono de corazón junto a un producto:

1. La app comprueba si ya está en favoritos.
2. Según el estado, añade o elimina un documento en la subcolección favouriteProducts de Firestore.
3. Actualiza la interfaz cambiando el icono para indicar si está marcado o no.

Al cargar la pantalla, si hay un usuario conectado, se ejecuta checkFavoriteState() para consultar en Firestore si existe un documento con el ID del producto y así ajustar el icono desde el inicio.

Comprobar el estado de 'Favoritos'

Cuando se carga ProductDetailActivity, si hay un usuario conectado, se comprueba en su subcolección favouriteProducts si existe un documento cuyo ID coincida con el del producto actual.

```
1 usage
private void checkFavoriteState() {
    DatabaseReference favRef = db
        .collection( collectionPath: "users") CollectionReference
        .document(uid) DocumentReference
        .collection( collectionPath: "favouriteProducts") CollectionReference
        .document(productId);

    favRef.get().addOnSuccessListener( DocumentSnapshot doc -> {
        isFavorite = doc.exists();
        updateHeartIcon();
    });
}
```

Ilustración 31 - Método de comprobar estado de favoritos

Añadir a favoritos

Cuando el usuario pulsa sobre el corazón y no está como favorito entonces la app llama a addToFavorites().

```
1 usage
private void addToFavorites() {
    DatabaseReference favRef = db
        .collection( collectionPath: "users") CollectionReference
        .document(uid) DocumentReference
        .collection( collectionPath: "favouriteProducts") CollectionReference
        .document(productId);

    Map<String,Object> data = new HashMap<>();
    data.put( k: "productId", productId);

    favRef.set(data).addOnSuccessListener( Void v -> {
        isFavorite = true;
        updateHeartIcon();
    }).addOnFailureListener( Exception e ->
        Toast.makeText( context: this, text: "Error al guardar favorito", Toast.LENGTH_SHORT).show()
    );
}
```

Ilustración 32 - Metodo de añadir a favoritos

Quitar de favoritos

Cuando el usuario pulsa el corazón relleno, lo que indica que ya está en favoritos, la app llama a `removeFromFavorites`.

```
private void removeFromFavorites() {  
    DatabaseReference favRef = db  
        .collection( collectionPath: "users") CollectionReference  
        .document(uid) DocumentReference  
        .collection( collectionPath: "favouriteProducts") CollectionReference  
        .document(productId);  
  
    favRef.delete().addOnSuccessListener( Void v -> {  
        isFavorite = false;  
        updateHeartIcon();  
    }).addOnFailureListener( Exception e ->  
        Toast.makeText( context: this, text: "Error al quitar favorito", Toast.LENGTH_SHORT).show()  
    );  
}
```

Ilustración 33 - Método de quitar un producto de favoritos

A continuación, podemos ver todos estos flujos en este diagrama:

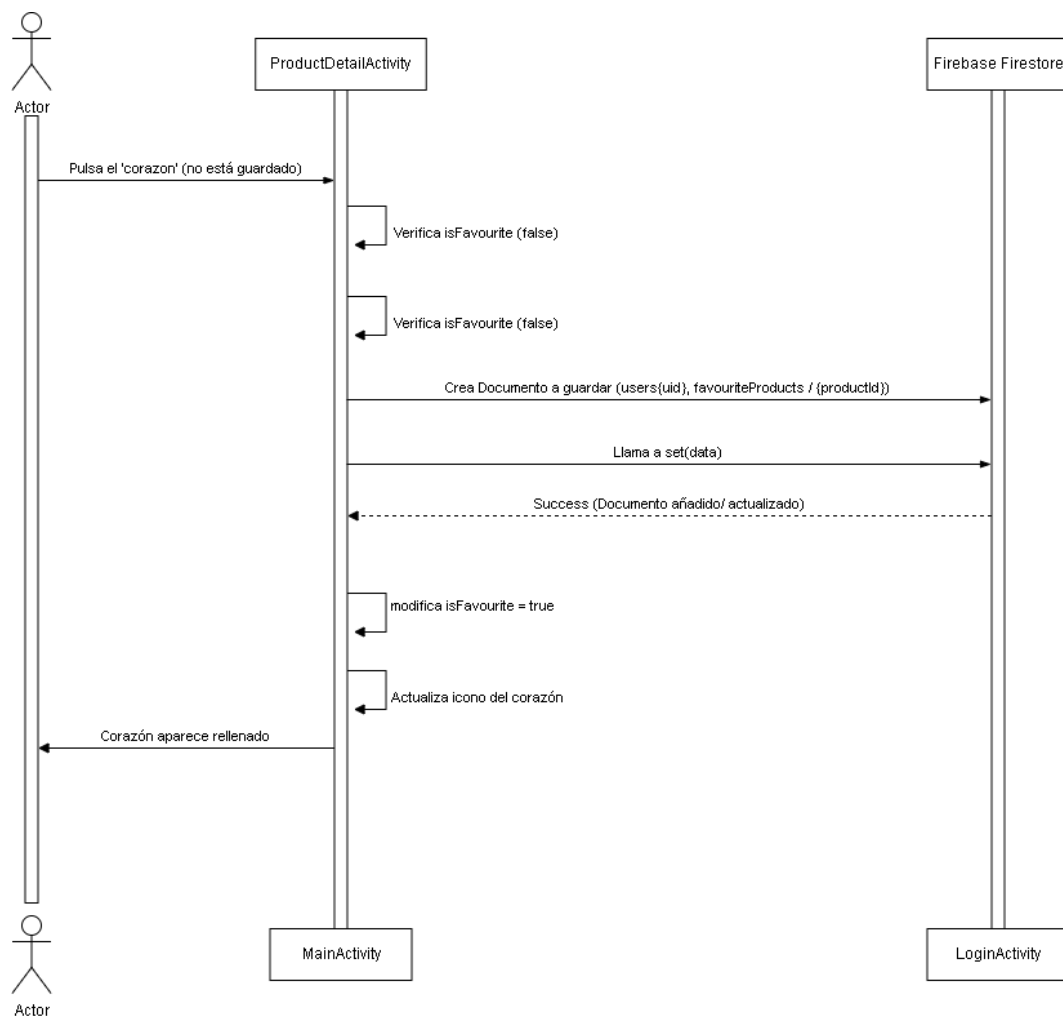


Ilustración 34 - Diagrama de flujo de los favoritos

8.4. Carga de datos y adaptadores

Cuando obtenemos datos de un producto de nuestra base de datos Firebase Firestore, no recibimos directamente un objeto `Product`. Recibimos algo llamado **DocumentSnapshot**. La clase `Product` actúa como un mapa para convertir fácilmente estos datos en bruto de la base de datos en un objeto estructurado con el que podemos trabajar.

Aquí podemos ver un trozo de código donde se utilizan estos documentos:

```
1 usage
private void searchProductByBarcode(String barcode) {
    db.collection( collectionPath: "products") CollectionReference
        .whereEqualTo( field: "barCode", barcode) Query
        .limit(1)
        .get() Task<QuerySnapshot>
        .addOnSuccessListener( QuerySnapshot queryDocumentSnapshots -> {
            if (!queryDocumentSnapshots.isEmpty()) {
                DocumentSnapshot doc = queryDocumentSnapshots.getDocuments().get(0);
                product = doc.toObject(Product.class);
                if (product != null) {
                    updateUI(product);
                    productId = product.getId();
                    FirebaseAuth.getInstance().getCurrentUser();
                    if (user != null) {
                        uid = user.getUid();
                        favoriteIcon.setVisibility(View.VISIBLE);
                    }
                }
            }
        })
}
```

Ilustración 35 - Método de buscar producto

Mostrar los datos de los productos

Cuando mostramos una lista de productos (por ejemplo, en `HomeActivity` o `FavoritesActivity`), usamos un Adaptador (como `ProductAdapter` o `FavoritesAdapter`; en el código actual, ambos son instancias de `ProductAdapter`). La misión del adaptador es tomar una lista de objetos `Product` y crear los elementos visuales (`item_product.xml`) para cada uno de ellos.

```
@Override
public void onBindViewHolder(@NonNull ProductViewHolder holder, int position) {
    Product product = products.get(position);
    holder.nameText.setText(product.getName());
    holder.brandText.setText(
        product.getBrandName() != null ? product.getBrandName() : "Desconocida"
    );
    holder.priceText.setText(
        product.getMinPrice() > 0
        ? product.getMinPrice() + " €"
        : ""
    );
    String pricePerUnit = product.getPricePerUnit();
    if (!pricePerUnit.equals("-")) {
        holder.pricePerUnitText.setText(pricePerUnit + " € / " + product.getUnit());
    } else {
        holder.pricePerUnitText.setText("");
    }
    String qty = (product.getQuantityPack() > 0 && product.getQuantityUnit() > 0)
        ? product.getQuantityPack() + " x " + product.getQuantityUnit() + " " + product.getUnit()
        : "";
    holder.quantityText.setText(qty);
    Glide.with(holder.itemView.getContext()) RequestManager
        .load(product.getPhotoUrl()) RequestBuilder<Drawable>
        .placeholder(R.drawable.ic_launcher_background)
        .error(R.drawable.ic_launcher_background)
        .into(holder.imageView);

    holder.supermarketLogoContainer.removeAllViews();
    for (String logoUrl : product.getSupermarketLogoUrls()) {...}
    holder.itemView.setOnClickListener( View v -> listener.onProductClick(product));
}
```

Ilustración 36 - Código del `productAdapter`

A continuación, veremos el flujo en este diagrama:

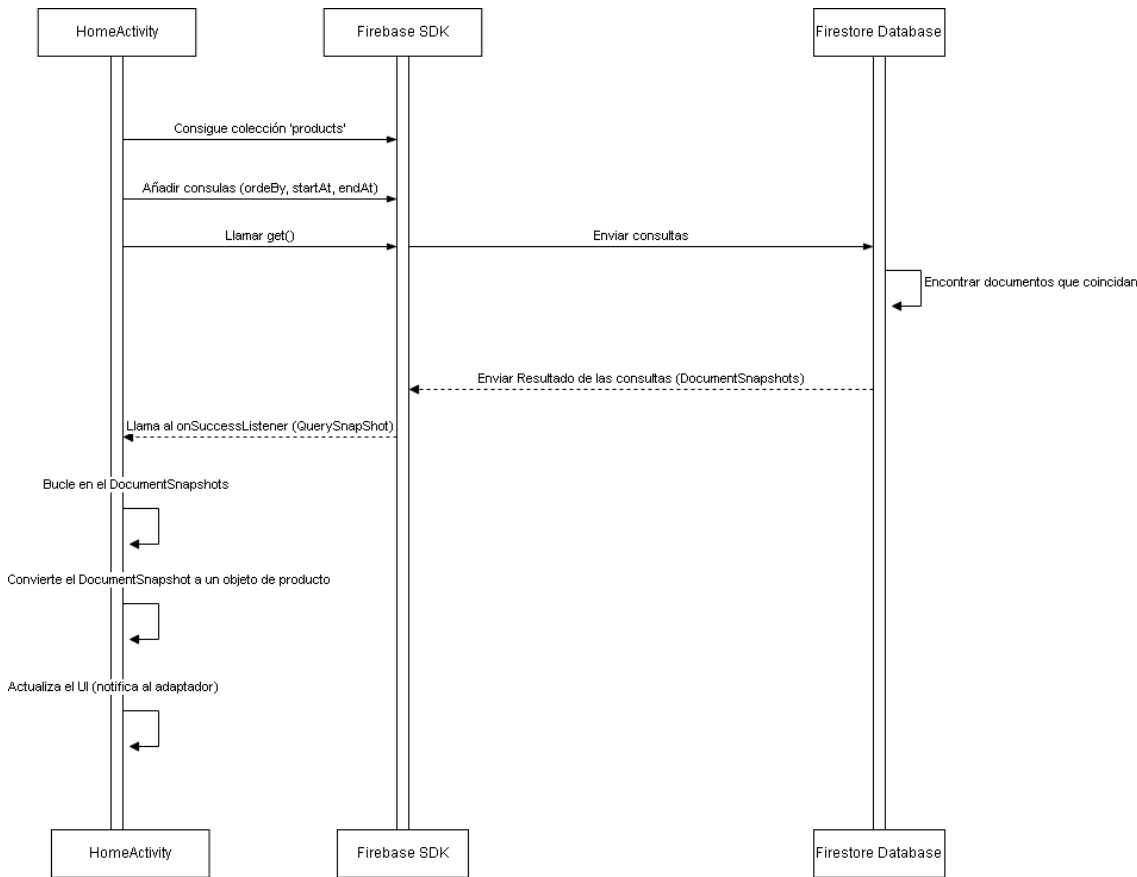


Ilustración 37 - Diagrama de flujo de la búsqueda de un producto

8.5. Lectura de datos: obtención de productos

Centremos un caso de uso clave: mostrar una lista de productos en la pantalla principal (HomeActivity). Esto implica leer datos de la colección “products”. A menudo, el usuario escribe algo en la barra de búsqueda, por lo que debemos consultar la base de datos para encontrar productos que coincidan con su texto de búsqueda.

Hacer una consulta en Firestore significa pedir al sistema documentos que cumplan ciertas condiciones.

```
public void searchProducts(String searchText) {
    if (searchText.isEmpty()) {
        labelSearchHistory.setVisibility(View.VISIBLE);
        historyRv.setVisibility(View.VISIBLE);

        productList.clear();
        adapter.notifyDataSetChanged();
        return;
    }
    labelSearchHistory.setVisibility(View.GONE);
    historyRv.setVisibility(View.GONE);

    db.collection(collectionPath: "products") CollectionReference
        .orderBy(field: "name") Query
        .startAt(searchText)
        .endAt(...fieldValues: searchText + "\uf8ff")
        .get() Task<QuerySnapshot>
        .addOnSuccessListener( QuerySnapshot productSnapshots -> {
            productList.clear();
            adapter.notifyDataSetChanged();
            for (DocumentSnapshot doc : productSnapshots.getDocuments()) {
                Product p = doc.toObject(Product.class);
                if (p == null) continue;
                p.setId(doc.getId());
                loadProductBrand(p, () -> {
                    loadPricesAndSupermarkets(p, () -> {
                        productList.add(p);
                        adapter.notifyDataSetChanged();
                    });
                });
            }
        })
        .addOnFailureListener( Exception e -> {
            Toast.makeText(context: this, text: "Error buscando productos", Toast.LENGTH_SHORT).show();
        });
}
```

Ilustración 38 - Método de búsqueda de producto

8.6. Escritura de datos

Además de leer, la app también necesita escribir datos en Firestore. Un ejemplo es añadir un nuevo producto, que es una función para usuarios administrador o moderador que se gestiona en AddProductActivity.java. Añadir un producto es algo más complejo porque implica varios documentos en distintas colecciones (products, productSupermarket y priceUpdate).

Cuando un usuario administrador rellena el formulario y pulsa “Guardar”, se llama al método saveProduct().

```
private void saveProduct() {
    String brandId = brandsList.get(spinnerBrand.getSelectedItemPosition()).getId();
    String typeId = typesList.get(spinnerType.getSelectedItemPosition()).getId();
    String supId = supsList.get(spinnerSupermarket.getSelectedItemPosition()).getId();

    Map<String, Object> prod = new HashMap<>();
    prod.put(k: "barCode", etBarCode.getText().toString().trim());
    prod.put(k: "name", etName.getText().toString().trim());
    prod.put(k: "brandId", brandId);
    prod.put(k: "productType", typeId);
    prod.put(k: "quantityPack", Double.parseDouble(etQtyPack.getText().toString().trim()));
    prod.put(k: "quantityUnity", Double.parseDouble(etQtyUnity.getText().toString().trim()));
    prod.put(k: "unit", etUnit.getText().toString().trim());
    String url = etPhotoUrl.getText().toString().trim();
    prod.put(k: "photoUrl", url);

    db.collection(collectionPath: "products") CollectionReference
    .add(prod) Task<DocumentReference>
    .addOnSuccessListener( DocumentReference docRef -> {
        String newProdId = docRef.getId();
        Map<String, Object> rel = new HashMap<>();
        rel.put(k: "productId", newProdId);
        rel.put(k: "supermarketId", supId);
        db.collection(collectionPath: "productSupermarket") CollectionReference
        .add(rel) Task<DocumentReference>
        .addOnSuccessListener( DocumentReference psRef -> {
            Map<String, Object> upd = new HashMap<>();
            upd.put(k: "price", Double.parseDouble(etInitialPrice.getText().toString().trim()));
            upd.put(k: "lastPriceUpdate", Timestamp.now());
            psRef.collection(collectionPath: "priceUpdate") CollectionReference
            .add(upd) Task<DocumentReference>
            .addOnSuccessListener( DocumentReference r -> {
                Toast.makeText(context: this, text: "Producto creado con éxito", Toast.LENGTH_SHORT).show();
                finish();
            })
            .addOnFailureListener( Exception e ->
                Toast.makeText(context: this, text: "Error al guardar precio inicial", Toast.LENGTH_SHORT).show()
            );
        });
    });
    .addOnFailureListener( Exception e ->
```

Ilustración 39 - Método de guardar nuevo producto en la base de datos

8.7. Lógica de comparación de precios

En `priceOnApp`, para mostrar el “Desde” de cada producto buscamos en Firestore todos los enlaces entre ese producto y los supermercados (colección `productSupermarket`). Para cada enlace, consultamos su subcolección `priceUpdate`, ordenamos por fecha de actualización descendente y tomamos la última entrada de precio. Comparamos estos precios recientes de cada supermercado para quedarnos con el mínimo, que es el que mostramos al usuario.

Opcionalmente, también obtenemos los datos del supermercado (como el logo) para indicar de dónde proviene ese precio. Todo esto se hace de forma asíncrona en métodos como `loadPricesAndSupermarkets` y `calculateMinPrices` de las Activities que muestran listas o detalles de productos.

```
public void loadPricesAndSupermarkets(Product product, Runnable onDone) {
    db.collection( collectionPath: "productSupermarket") CollectionReference
        .whereEqualTo( field: "productId", product.getId()) Query
        .get() Task<QuerySnapshot>
        .addOnSuccessListener( QuerySnapshot productSuperDocs -> {
            if (productSuperDocs.isEmpty()) {
                onDone.run();
                return;
            }
            AtomicInteger processed = new AtomicInteger( initialValue: 0);
            double[] min = { Double.MAX_VALUE };
            String[] cheapestSuperId = { null };

            for (DocumentSnapshot superDoc : productSuperDocs) {
                String psId = superDoc.getId();
                String supermarketId = superDoc.getString( field: "supermarketId");
                db.collection( collectionPath: "productSupermarket") CollectionReference
                    .document(psId) DocumentReference
                    .collection( collectionPath: "priceUpdate") CollectionReference
                    .orderBy( field: "lastPriceUpdate", Query.Direction.DESENDING) Query
                    .limit(1)
                    .get() Task<QuerySnapshot>
                    .addOnSuccessListener( QuerySnapshot priceDocs -> {
                        for (DocumentSnapshot pd : priceDocs) {
                            Double price = pd.getDouble( field: "price");
                            if (price != null && price < min[0]) {
                                min[0] = price;
                                cheapestSuperId[0] = supermarketId;
                            }
                        }
                    })
                db.collection( collectionPath: "supermarkets") CollectionReference
                    .document(supermarketId) DocumentReference
                    .get() Task<DocumentSnapshot>
                    .addOnSuccessListener( DocumentSnapshot sDoc -> {
                        String logoUrl = sDoc.getString( field: "logoUrl");
                        if (logoUrl != null) product.addSupermarketLogoUrl(logoUrl);

                        if (processed.incrementAndGet() == productSuperDocs.size()) {
                            if (min[0] < Double.MAX_VALUE) {
                                product.setMinPrice(min[0]);
                                product.setCheapestSupermarketId(cheapestSuperId[0]);
                            }
                        }
                    })
            }
        })
    onDone.run();
}
```

Ilustración 40 - Método de calcular precio por supermercado

8.8. Funcionalidad del lector de código de barras

Añadimos la función de escaneo de códigos de barras usando **CameraX** y **ML Kit** para que el usuario, al apuntar con la cámara al código de un producto, lo detecte, extraiga el número y lo busque automáticamente en Firestore. Creamos una **BarcodeScannerActivity** con un **PreviewView** que muestra el vídeo en directo y configuramos CameraX para enviar cada fotograma al escáner de ML Kit.

Al detectar un código válido, la app lanza un Intent a `ProductDetailFromBarcodeActivity`, donde se realiza una consulta `whereEqualTo("barCode", barcode)` en la colección de productos, se obtiene el documento correspondiente y se muestran sus detalles y precios agregados. De este modo, el usuario puede comprobar al instante el precio de cualquier artículo sin escribir nada.

Primero creamos la vista del escanner:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.camera.view.PreviewView
        android:id="@+id/previewView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toTopOf="@id/bottomNavigationBar"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottomNavigationBar"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:background="@color/my_dark_primary"
        app:menu="@menu/bottom_navigation_menu"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Ilustración 41 -XML de lector de código de barras

Integramos **CameraX** y el **ML Kit** de la siguiente manera:

1. Solicitar permiso para usar la cámara.
2. Configurar CameraX para mostrar la vista previa y proporcionar imágenes para su análisis.
3. Configurar el BarcodeScanner de ML Kit para procesar esas imágenes.
4. Definir qué ocurre cuando ML Kit detecta un código de barras.

```
private void bindPreviewAndAnalyzer(@NonNull ProcessCameraProvider cameraProvider) {
    cameraProvider.unbindAll();
    androidx.camera.core.Preview preview = new androidx.camera.core.Preview.Builder().build();
    preview.setSurfaceProvider(previewView.getSurfaceProvider());
    CameraSelector cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA;
    ImageAnalysis imageAnalysis = new ImageAnalysis.Builder()
        .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
        .build();
    BarcodeScannerOptions options =
        new BarcodeScannerOptions.Builder()
            .setBarcodeFormats(
                Barcode.FORMAT_EAN_13,
                Barcode.FORMAT_EAN_8,
                Barcode.FORMAT_UPC_A,
                Barcode.FORMAT_CODE_39,
                Barcode.FORMAT_CODE_128)
            .build();

    BarcodeScanner scanner = BarcodeScanning.getClient(options);
    imageAnalysis.setAnalyzer(cameraExecutor, ImageProxy imageProxy -> {
        if (barcodeDetected) {
            imageProxy.close();
            return;
        }
        processImageProxy(scanner, imageProxy);
    });
    cameraProvider.bindToLifecycle(lifecycleOwner: this, cameraSelector, preview, imageAnalysis);
}
```

Ilustración 42 - Método que procesa el lector de código de barras

9. Resultados finales

9.1. Pantallas iniciales y de autenticación

Aquí podemos ver como ha quedado las pantallas de autenticación del usuario y la vista principal de la pantalla.

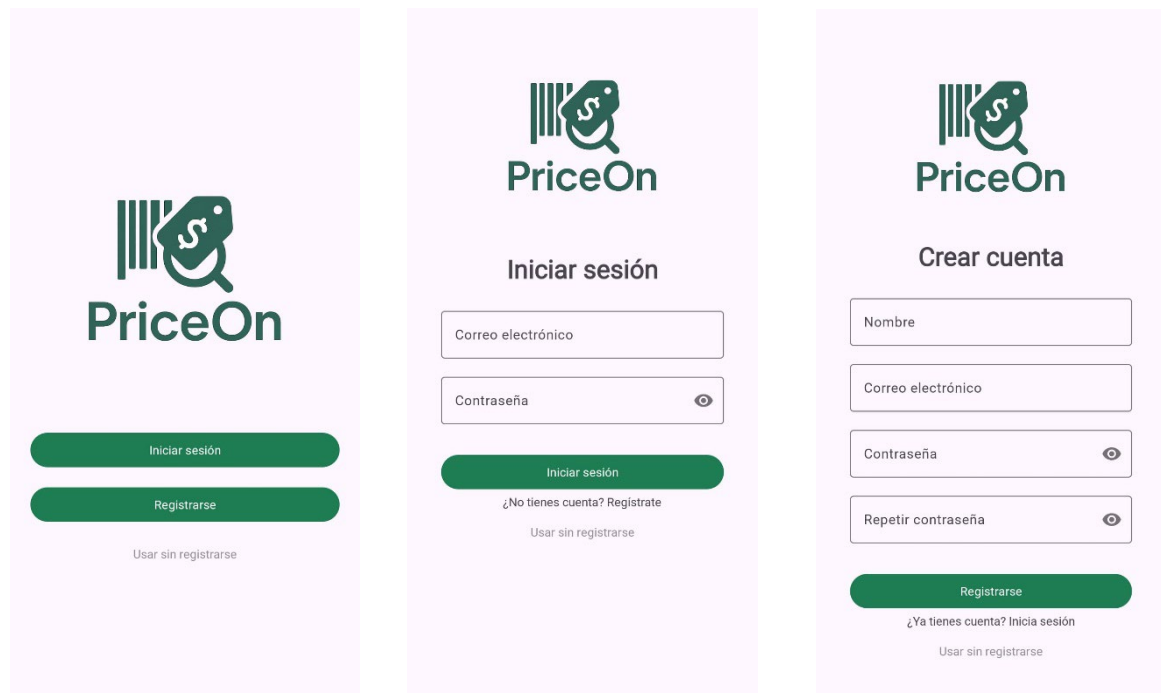


Ilustración 43 - Pantallas finales de autenticación

9.2. Página de inicio y perfil de usuario

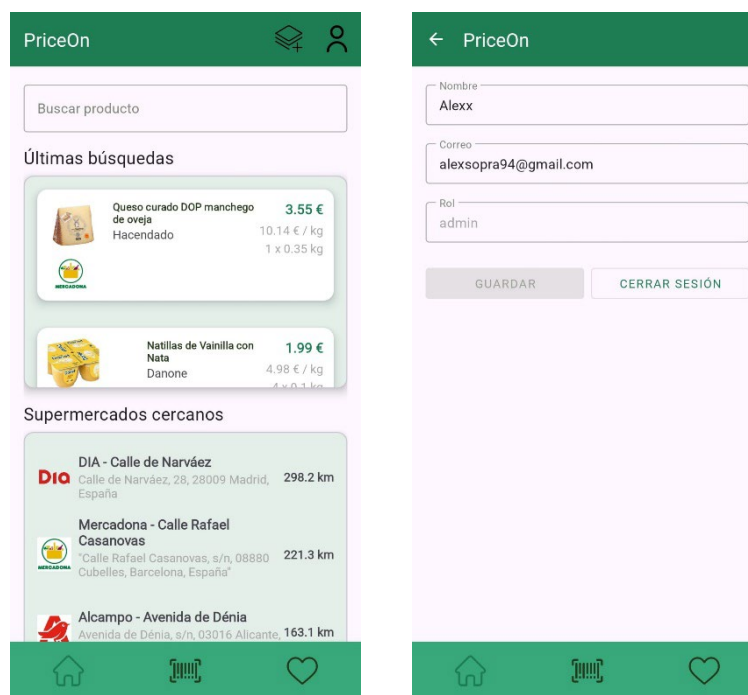


Ilustración 44 - Pantalla inicial y perfil de usuario

9.3. Pantalla de búsqueda y detalles de producto

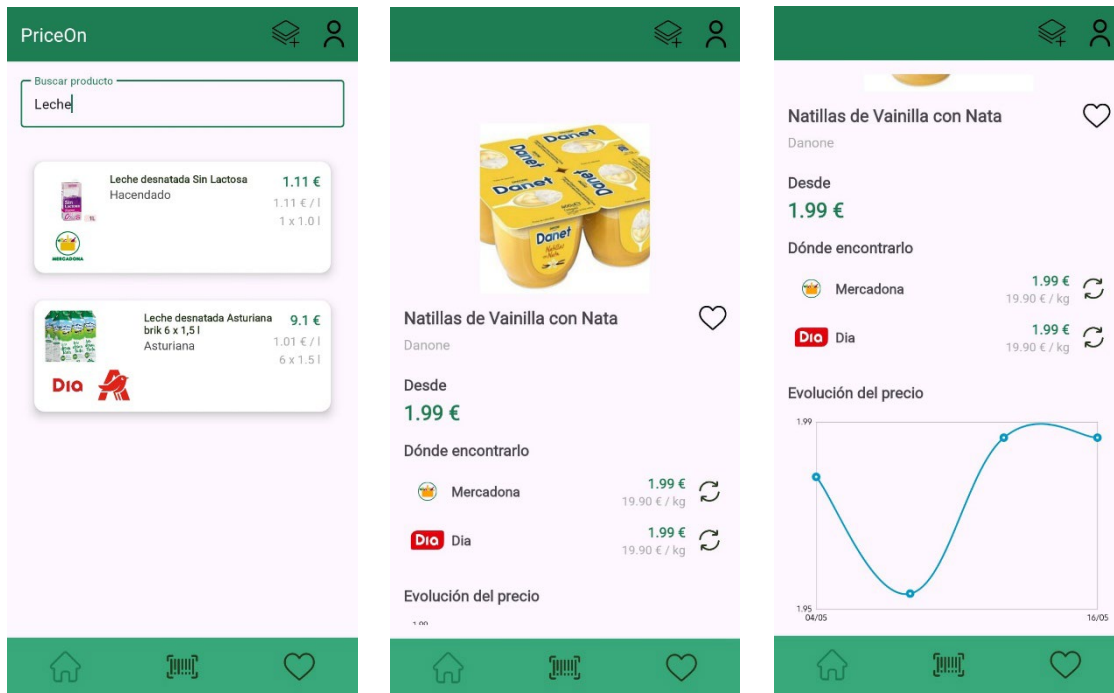


Ilustración 45 - Pantallas de búsqueda de producto

9.4. Pantallas de favoritos, actualización de precio

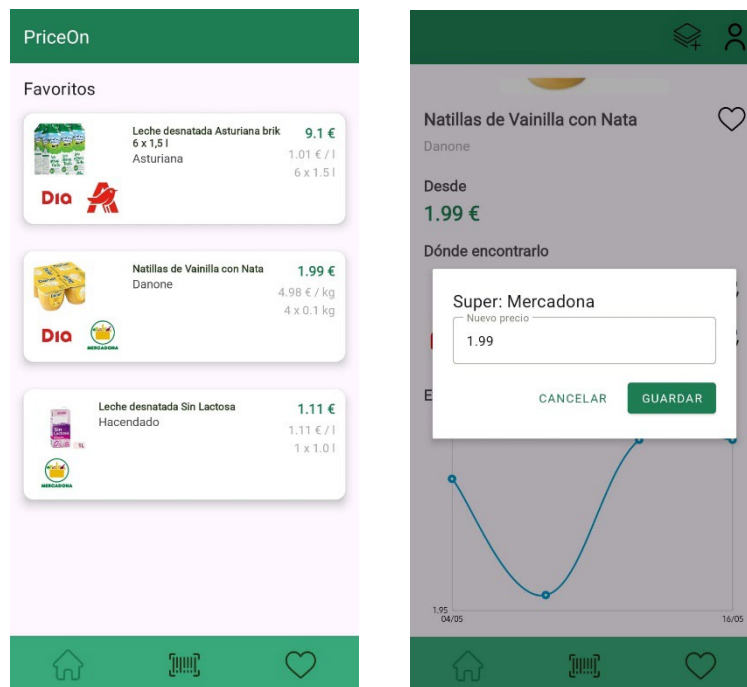


Ilustración 46 - Pantallas de favoritos y actualización de precio

9.5. Pantalla de añadir nuevo producto

Nuevo producto

Código de barras

Nombre del producto

URL de la foto

Marca
Nestlé

Tipo de producto
Conservas

Cantidad pack

Cantidad unidad

Unidad (e.g. L, kg)

Supermercado
Dia



Nuevo producto

URL de la foto

Marca
Nestlé

Tipo de producto
Conservas

Cantidad pack

Cantidad unidad

Unidad (e.g. L, kg)

Supermercado
Dia

Precio inicial

GUARDAR PRODUCTO



Ilustración 47 - Pantallas de nuevo producto

10. Conclusiones y Trabajo Futuro

10.1. Evaluación del grado de cumplimiento de los objetivos

En general, el proyecto ha cumplido el 100% de los objetivos planteados:

- Nuestra app puede **buscar productos** por nombre y código de barras.
- **Comparativa y evolución de precios**: la lógica de agregación de precios en Firestore muestra correctamente el precio más bajo, el precio en todos los supermercados y una evolución en una gráfica.
- **Registro de usuarios, favoritos e historico**: hemos implementado funcionalidades como el registro de usuarios y la gestión de perfil, esto necesario para poder implementar una lista de favoritos e historico de visualizaciones de nuestros usuarios.
- **Modelo colaborativo y open-source**: el código estará disponible en GitHub bajo licencia MIT y soporta contribuciones externas, cumpliendo el enfoque ético y comunitario del proyecto.

10.2. Análisis de la Planificación

Analizando el diagrama Gantt final (anexo [1]), podemos ver varias diferencias.

Inicialmente, se planteó llevar a cabo el desarrollo de forma continua y progresiva siguiendo una metodología ágil, con sprints periódicos en los que se entregarían funcionalidades completas, e ir incorporando nuevas características según lo permitiera el calendario.

En la práctica, no fue posible aplicar la metodología ágil prevista y el desarrollo siguió un modelo en cascada: primero se definieron por completo requisitos y diseño, luego se implementaron los módulos principales (autenticación, agregación de precios y, al final, integración con cámara y ML Kit) y finalmente se realizó la documentación. Este enfoque casi no garantiza el cumplimiento de plazos, y redujo la flexibilidad, retrasó la detección de errores y encareció los cambios de última hora.

10.3. Lecciones aprendidas durante el desarrollo

He aprendido varias lecciones en todo este proceso. Una de ellas y creo que la más importante: creer en mi misma. Este proceso me ha demostrado que con persistencia y trabajo duro puedo conseguir todo lo que me proponga, aunque cueste lo suyo, el resultado final ha valido la pena.

Ahora que nos hemos quitado lo sentimental, podemos ver las lecciones que nos ha enseñado el código:

- **Importancia de la planificación ágil**: incorporar revisiones periódicas (sprints cortos) habría permitido detectar antes los cuellos de botella en integración con Firebase y ML Kit.
- **Gestionar las dependencias asíncronas**: la complejidad de las llamadas aninadas a Firestore y ML Kit resaltó la necesidad de patrones como Coroutines o callbacks mejor estructuradas para evitar el famosísimo “callback hell”

10.4. Posibles mejoras y líneas de trabajo futuras.

Almacenamiento de imágenes en Realtime Database o Cloud Storage

Actualmente las fotos de los productos se sirven mediante URLs fijas, lo que complica la gestión y puede generar enlaces rotos. Una mejora sería migrar la gestión de imágenes a Firebase Realtime Database (o, preferiblemente, a Firebase Cloud Storage junto a Realtime Database para los metadatos).

Gestión dinámica de roles y peticiones de moderación

Se podría implementar un panel de administración para que los administradores puedan:

- Visualizar y modificar el rol de cualquier usuario
- Recibir y gestionar solicitudes de ascenso de rol formuladas por usuarios desde la app.

Flujo de revisión previa a la actualización de precios

Para garantizar la fiabilidad de los cambios de precio, introducir un mecanismo de revisión. Cuando un usuario envía un nuevo precio, éste se almacena en una subcolección “pendingPriceUpdates” en lugar de aplicarse directamente.

A continuación un moderador o administrador recibirá una alerta para validar la coherencia del precio.

Integrar fuentes de datos externas para los precios

El modelo actual depende de aportes manuales de los usuarios, lo cual es lento y poco fiable. Para acelerar y robustecer la obtención de precios, se podrían integrar APIs de terceros o web scraping automático de las webs de cadenas de supermercados

11. Bibliografía y Referencias

- [1] Cinco Días. (25 de octubre de 2024). «La crisis inflacionaria dispara el negocio de los grandes supermercados regionales». Cinco Días. Available: <https://cincodias.elpais.com/companias/2024-10-25/la-crisis-inflacionaria-dispara-el-negocio-de-los-grandes-supermercados-regionales.html>
- [2] El Debate. (12 de abril de 2025). «Reduflación resiste a la ley de “menos cantidad”: debate cero transparencia». El Debate. Available: https://www.eldebate.com/economia/20250412/reduflacion-resiste-ley-menos-cantidad-debate-cero-transparencia_287688.html
- [3] El Androide Libre. (3 de mayo de 2024). «Una sencilla acción para ver y comparar precios de supermercado en el móvil antes de ir de compra». El Androide Libre. Available: https://www.lespanol.com/elandroidelibre/aplicaciones/20240503/sencilla-accion-ver-comparar-precios-supermercado-movil-ir-hacer-compra/844415833_0.html
- [4] Comparaloo. (s. f.). Comparaloo [Sitio web]. <https://comparaloo.es>
- [5] Soysuper. (2025). Soysuper [Aplicación Android]. Google Play. <https://play.google.com/store/apps/details?id=com.soysuper.soysuper&hl=es>
- [6] Findit. (2025). Findit [Aplicación Android]. Google Play. <https://play.google.com/store/apps/details?id=com.findit.findit&hl=en>
- [7] Oracle. Java Platform, Standard Edition Documentation. Oracle. <https://docs.oracle.com/en/java/javase/>
- [8] Android Developers. Android Studio. Google. <https://developer.android.com/studio>
- [9] World Wide Web Consortium (W3C). Extensible Markup Language (XML) 1.0 (Fifth Edition). W3C, <https://www.w3.org/TR/xml/>
- [10] Firebase. Firebase Authentication Documentation. Google. <https://firebase.google.com/docs/auth>
- [11] Firebase. Cloud Firestore Documentation. Google. , <https://firebase.google.com/docs/firestore>
- [12] Google Developers. ML Kit Barcode Scanning. Google. , <https://developers.google.com/ml-kit/vision/barcode-scanning>
- [13] Philipp Jahoda. MPAndroidChart Documentation. <https://weeklycoding.com/mpandroidchart-documentation/>
- [14] GitHub. GitHub Docs. GitHub. <https://docs.github.com/>
- [15] Bumptech. Glide: Fast and Efficient Image Loading for Android. <https://bumptech.github.io/glide/>
- [16] Android Developers. CameraX Overview. Google. <https://developer.android.com/media/camera/camerax>

12. Anexo

[1] Diagrama de Gantt Final

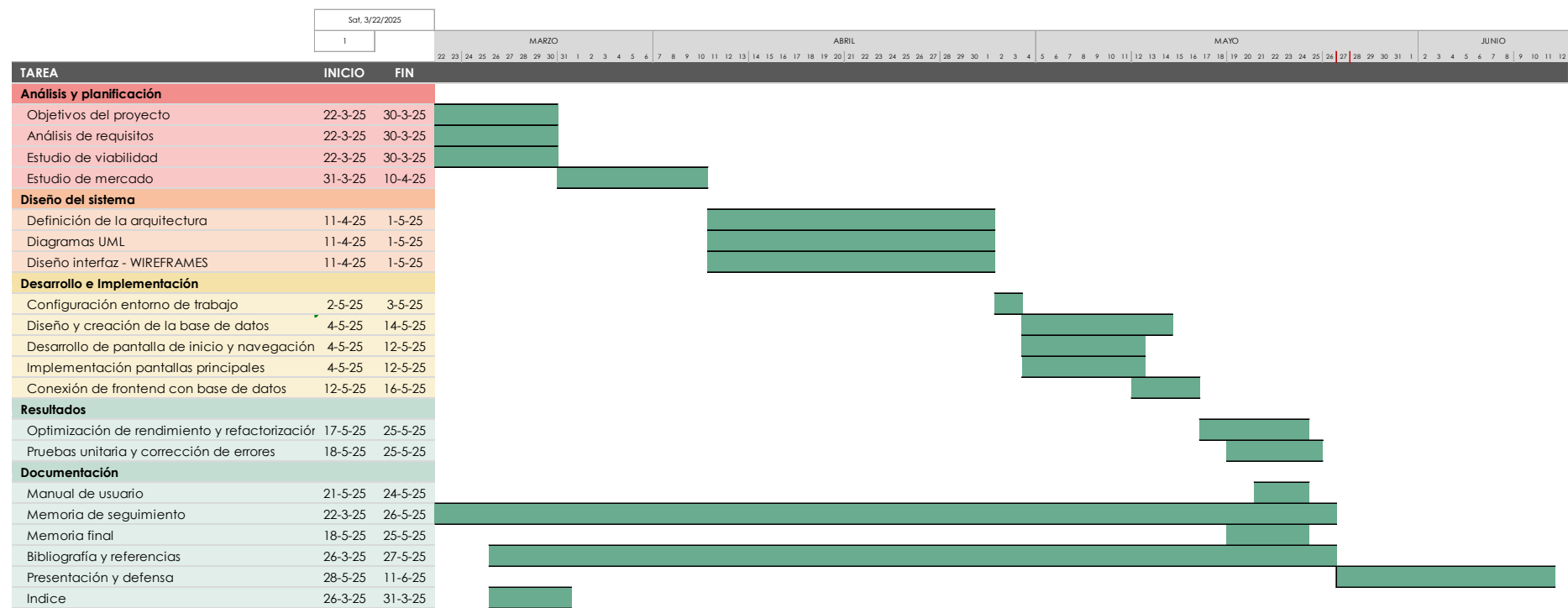


Ilustración 48 - Diagrama de Gantt Final

Firmado por RALUCA ALEXANDRA SAVU - NIF:****8038* el día 27/05/2025 con un certificado emitido por ACCV RSA1 CLIENTE